claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_887fb14e-e6d\agent-aefc97bff8c3710b4.jsonl`
- SHA-256: `095ef7f4614fa81fd71571231a37f6bd0910aadff8e284770541312dd26f92d8`
- Source modified: `2026-06-20T10:12:41+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_887fb14e-e6d`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`

Transcript

1. user (2026-06-20T10:03:00.507Z)

Goal: design dedicated Studio-UI pages that turn "annotate your footage ? train your OWN model" into a first-class USP (the bring-your-own-model moat), linking the public GitHub repo (https://github.com/avidullu/rally-annotator), with screenshots and step-by-step setup/use help. Be HONEST about what is real today vs roadmap. Tag claims [verified]/[design]/[researched]. Do NOT invent capabilities. EXCLUDE any .claude/worktrees/ paths.

Design the USP framing + user setup help (read C:\\Users\\avidu\\Projects\\khelsutra/site/public/index.html for current positioning + honesty sections; C:\\Users\\avidu\\Projects\\badminton-highlight-indexer/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md + CLAUDE.md guardrails for the commercial-clean / minors-excluded / paid-LLM-never-a-training-source rules; C:\\Users\\avidu\\Projects\\badminton-highlight-indexer/docs/ROORA_LABELING_GUIDELINES.md for labeling guidance). Produce: an honest USP framing for "annotate & train your OWN model" as the moat (contrast with paid LLMs you cannot train on your data; tie to privacy/consent + commercial-clean licensing + minors-excluded); the end-to-end setup-help outline (install VLC plugin ? record ? annotate ? export CSV ? train ? use); a screenshot plan (what to show + source: annotator-repo images vs to-capture ? NEVER fabricate); and honest caveats (train is GPU/native-WASB-gated; annotator is a separate VLC tool, not in-browser; manual steps). Return the schema.

2. assistant (2026-06-20T10:03:03.652Z)

I'll research the current state of the codebase, the annotator repo, and the positioning docs to design an honest USP framing. Let me start by reading the key files in parallel.

3. user (2026-06-20T10:03:05.073Z)

```
1	# Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3	**Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner greenlights a specific
4	milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a training source).
5	**Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO + learned prototype
6	(`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy workflow (2026-06-09).
7
8	## Design principle (owner directive): *envision now, build later*
9	Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be **architecturally envisioned
10	now** so we never hit a mammoth refactor. We **design the seams** (data contracts, export boundaries, the
11	event store) up front and **implement behind them** when each milestone is greenlit.
```

Punting a *feature* is fine; punting its *seam* is not.

North Star (reframed per PR #61 review)
- **Immediate deliverable:** a **training framework + harness** that produces a model with **very high precision/ recall for extracting highlights + rallies from a clip**. *Quality is the first-order objective.*
- **Eventual product:** a **single, sport-agnostic, conversational** video model ? upload a clip, then ask contextual questions ("longest rally", "make a clip of all service faults", "rallies over 20 shots").
- **On-device is LATER** (a model-compression goal so power users save upload cost/hassle) ? definitely on the roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
- **The moat = the consented dataset + the eval harness, not the weights.**

Hard guardrails (non-negotiable ? "never corrupted")
- **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT / Apache-2.0 / BSD only.** (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-restrictive: Gemma 3 (viral), **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU cap), Commons-Clause repos.
- **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision (a real, binding cost).
- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**
- ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl. Qwen3-VL).** Our "no paid-LLM training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ? an **explicit owner risk-acceptance** is required to use any open VLM. (Governed by the base's license, not our pipeline.)

The model strategy (verified 2026-06-09)
Framework ? base (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge (runs SFT/GRPO, emits weights); a *base* is the metal you forge. You pick one framework and feed it any clean base.
- **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio + timestamp-grounding + GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core; *avoid its AGPL Studio UI*) as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-time snapshot ? re-verify at impl.
- **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-video context, purpose-built **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm the exact size's LICENSE pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but its Prohibited-Use posture is unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not the primary extractor. **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is cheap *only within timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).
- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries** (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries"). **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-

risked on our corpus (held-out AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later), and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**, strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)*

Conversational video-QA ? build the bookkeeping NOW, punt the model
Punt the conversational *feature* until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn** (expensive; bad at exact counting; the strong teachers are license-forbidden). **But build the *seam* now** (the brief requires it):
- **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits structured timestamped spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-SQL + ffmpeg clip-cut). The cited queries ("longest rally", "all service faults", "rallies > 20 shots") are **structured-numeric ? SQL, not a VLM job** (SQL counts exactly where VLMs hallucinate).
- **Designed-now seam:** make the **per-video event-index schema a first-class data contract** alongside the M0.1 corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags, derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy (e.g. `service_fault`) up front** even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration): `play_segments` today has `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights` isn't a table yet ? so it's an *extension with new columns + a highlights table*, not a no-op. **Punt** the NL parser/agent until quality clears the bar.
- Paid Gemini may answer genuinely open-ended edge questions **at inference** over our structured index (allowed ? it reasons over our data, it is not trained on its outputs).

Multisport (single sport-agnostic model)
- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION: rename the repo off "badminton" ASAP** (owner emphatic) ? consistent with this.
- **What multisport changes = DATA + DETECTORS (scale up, the accepted "good problem"):** each new sport needs its own labeled matches (split by match) + a **clean trajectory detector** to feed the head's features. **The binding blocker is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-provenance issue, and **clean MIT/Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet identified.** Sequence by public-data richness: **badminton ? tennis/TT ? pickleball/padel.**
- ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not proven ? our corpus is badminton-only, and the evidence once cited for it (RacketVision) was *refuted on verification* (it's ball-tracking, not temporal). The concept is sound on first principles (rally signal = a racquet-sport invariant) but must be validated once a 2nd-sport corpus exists. Don't cite RacketVision as evidence.

Language ? Python everywhere; on-device runtime is a deferred, layer-local export shell

- **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer); offline/batch, no latency hot path.
- **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-process (ffmpeg, WASB-in-WSL, cloud
  GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ? single-digit-ms wins dwarfed by
  ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval harness + provider registry.
- **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate our dev language:** train in
  Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device shell (Swift/Kotlin/C++/Rust)
  runs the artifact with zero Python. **The one concrete constraint now: train with export-clean ops** so the compression
  target stays reachable without re-architecting. (On-device base stays on the Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
- Escalation order if a real Python CPU hot path ever appears: free-threading ? numpy/vectorize ? Cython/PyO3 ? only then a
  separate service. **None exists today.**

Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)

M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) · *greenlight item*
Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index schema (the conversational seam)
? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end, ending_reason/fault_tags,
shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY MATCH), trajectory_ref,
labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL ActivityNet-JSON · VLM clip?QA),
one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector` (system-of-record; extends PR #13).
Reserve fault/event taxonomy now. Acceptance: round-trip test; split-by-match enforced; **no Gemini-sourced rows in any training view**.

M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-match + LOVO recorded; the
"contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club match ? margin. **Bottleneck =
data + calibration, not method.** Keep growing per new sport.

M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) · *greenlight item*
- **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training violation is
  `backend/eval/distill_local.py` (it trains a classifier on Gemini-silver CSV labels ? `output/local_rally_model.json`)
  and the threshold distillation in `calibrate_local.py`. (NB: `backend/eval/training.py::label_candidates` is a *pure,
  source-agnostic* function ? there is no store by that name; the earlier "training.label_candidates" pointer was wrong.)
  Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the path that regenerates them is one
  command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini = eval/reference/fallback only.
- **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint provenance is a **commercial-ship
  blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a clean MIT/Apache detector swap.
  ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our

temporal labels can't train the
  shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies per sport** (incl. any TT-on-WASB use).
  Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).

M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over **ActionFormer (MIT, 1:1
rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not* the Commons-Clause repo),
reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item): a FLOOR of "beat the honest n=6
heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU** target for "very high P/R" (e.g.
F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains on the GPU/WSL box.

Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it becomes the rally/highlight
extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no overfitting to declare victory.

Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit go)
- **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a promising cue here** ? our E1 probe
  (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea also failed this session; keep it
  parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet first).
- **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a temporal-IoU-vs-golden reward,
  our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on Modal (no-access) / RunPod-Secure.
- **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-hosted VLM vs Gemini) **+ a real
  VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples use 6?8 GPUs ? **budget is the
  largest open risk and is currently unquantified**). Don't stand up an owned serving stack until these clear.

Decisions / status
#	item	status
1	Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4 harness**	awaiting owner
2	WASB C3 ? first-class action item (own-weights / clean swap)	**owner: yes** ?
3	Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights)	**owner: yes** ?
4	Collector = corpus + event-store system-of-record	**owner: confirm?**
5	Rename repo off "badminton"	**owner action** ?
6	Conversational-QA: build seam now, punt model	**rec accepted?**

Open risks / unknowns (none block Phase-0 research)
Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R" target undefined · per-sport clean
trajectory detectors not yet identified · base-model pretraining provenance unauditable (risk-acceptance) · real VLM
VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-train (minors-excluded) for cloud

training · on-device 4B latency on 8 GB unmeasured.

Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing discipline holds: the committed next platform slice remains the async job model (single-tenant); only the M0.1?M0.4 foundation is safe to start.

4. user (2026-06-20T10:03:05.207Z)

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>KhelSutra ? every rally indexed, the dead time gone</title>
<meta name="description" content="KhelSutra turns a full badminton match into a clean index of rallies with the dead time stripped out ? a highlight reel you can watch and download. Invite-only alpha, Bengaluru." />
<meta property="og:title" content="KhelSutra ? every rally indexed, the dead time gone" />
<meta property="og:description" content="A full match becomes a clean highlight reel ? only the real play, no dead time. Invite-only alpha for academies and players in Bengaluru." />
<meta property="og:type" content="website" />
<meta property="og:url" content="https://khelsutra.guru" />
<meta property="og:image" content="https://khelsutra.guru/og.png" />
<meta property="og:image:width" content="1200" />
<meta property="og:image:height" content="630" />
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="KhelSutra ? every rally indexed, the dead time gone" />
<meta name="twitter:description" content="A full match becomes a clean highlight reel ? only the real play, no dead time. Invite-only alpha, Bengaluru." />
<meta name="twitter:image" content="https://khelsutra.guru/og.png" />
<meta name="theme-color" content="#0C261F" />
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'%3E%3Crect width='32' height='32' rx='8' fill='%230C261F'/%3E%3Cpath d='M13 10 L23 16 L13 22 Z' fill='%2319B36B'/%3E%3C/svg%3E" />
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap" rel="stylesheet" />
<style>
  :root{
    --ink:#0C261F; --ink2:#123A2E; --green:#19B36B; --green-d:#149A5C;
    --lime:#C7F2B0; --paper:#F6F8F4; --card:#fff;
    --tx:#14271F; --mut:#566B61; --line:#E4E9E2;
    --on-dark:#EAF3EE; --on-dark-mut:#A7C3B7;
    --r:14px; --rs:10px; --maxw:1080px;
    --font:'Plus Jakarta Sans',system-ui,-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;
  }
  *{box-sizing:border-box}
  html{scroll-behavior:smooth}
  body{margin:0;font-family:var(--font);color:var(--tx);background:var(--paper);line-height:1.65;-webkit-font-smoothing:antialiased}
  a{color:inherit}
  .wrap{max-width:var(--maxw);margin:0 auto;padding:0 22px}
  h1,h2,h3{line-height:1.15;margin:0;letter-spacing:-.02em}
  h1{font-size:clamp(2rem,5vw,3.4rem);font-weight:800}
  h2{font-size:clamp(1.5rem,3.2vw,2.2rem);font-weight:700}
  h3{font-size:1.15rem;font-weight:700}
  p{margin:.6rem 0}
  .btn{display:inline-flex;align-items:center;gap:.5rem;font-weight:600;font-
```

```
size:1rem;border-radius:999px;padding:.78rem 1.4rem;text-decoration:none;border:1px solid transparent;cursor:pointer;transition:.15s}
    .btn-primary{background:var(--green);color:#04140D}
    .btn-primary:hover{background:var(--green-d)}
    .btn-ghost{background:transparent;border-color:rgba(255,255,255,.28);color:var(--on-dark)}
    .btn-ghost:hover{border-color:rgba(255,255,255,.6)}
    .btn-dark{background:var(--ink);color:#fff}
    .btn-dark:hover{background:var(--ink2)}
    .eyebrow{display:inline-flex;align-items:center;gap:.5rem;font-size:.82rem;font-weight:600;letter-spacing:.02em;
      background:rgba(25,179,107,.14);color:#0a6c45;border:1px solid rgba(25,179,107,.3);padding:.3rem .8rem;border-radius:999px}

    /* header */
    header{position:sticky;top:0;z-index:20;background:rgba(246,248,244,.86);backdrop-filter:blur(10px);border-bottom:1px solid var(--line)}
    .nav{display:flex;align-items:center;justify-content:space-between;height:66px}
    .brand{display:flex;align-items:center;gap:.6rem;font-weight:800;font-size:1.18rem;letter-spacing:-.02em;text-decoration:none}
    .brand .dot{color:var(--green)}
    .nav-links{display:flex;align-items:center;gap:1.6rem}
    .nav-links a{font-size:.95rem;font-weight:500;color:var(--mut);text-decoration:none}
    .nav-links a:hover{color:var(--tx)}
    .brand .play{display:inline-block;width:0;height:0;border-style:solid;border-width:7px 0 7px 12px;border-color:transparent transparent transparent var(--green);margin:0 .32em;transform:translateY(1px)}

    /* hero */
    .hero{background:linear-gradient(160deg,var(--ink) 0%,var(--ink2) 100%);color:var(--on-dark);position:relative;overflow:hidden}
    .hero .wrap{padding-top:74px;padding-bottom:88px;position:relative;z-index:2}
    .hero .eyebrow{background:rgba(199,242,176,.12);color:var(--lime);border-color:rgba(199,242,176,.25)}
    .hero h1{margin:1.1rem 0 0;max-width:15ch}
    .hero h1 .hl{color:var(--lime)}
    .hero .lead{font-size:clamp(1.05rem,1.6vw,1.28rem);color:var(--on-dark-mut);max-width:54ch;margin:1.2rem 0 0}
    .hero-cta{display:flex;flex-wrap:wrap;gap:.8rem;margin-top:2rem}
    .hero-note{margin-top:1.1rem;font-size:.86rem;color:var(--on-dark-mut)}
    .shuttle{position:absolute;right:-60px;top:-40px;width:520px;max-width:55%;opacity:.16;z-index:1;pointer-events:none}
    .loop{display:flex;flex-wrap:wrap;gap:.6rem;margin-top:2.4rem}
    .loop .chip{display:flex;align-items:center;gap:.5rem;background:rgba(255,255,255,.06);border:1px solid rgba(255,255,255,.12);
      border-radius:999px;padding:.5rem 1rem;font-weight:500;font-size:.92rem;color:var(--on-dark)}
    .loop .chip b{color:var(--lime);font-weight:700}
    .loop .arrow{color:var(--on-dark-mut);align-self:center}

    .hero-grid{display:grid;grid-template-columns:1.04fr .96fr;gap:46px;align-items:center}
    .hero-left{min-width:0}
    .hero-visual{min-width:0}
    .reel-card{background:#fff;border-radius:18px;border:1px solid rgba(255,255,255,.14);box-shadow:0 26px 60px -24px rgba(0,0,0,.55);overflow:hidden}
    .reel-head{display:flex;align-items:center;gap:.55rem;padding:.7rem .9rem;border-bottom:1px solid var(--line)}
    .reel-head .dots{display:flex;gap:5px}
    .reel-head .dots i{width:9px;height:9px;border-radius:50%;background:#D9DED7;display:block}
    .reel-head .file{font-size:.78rem;color:var(--mut);font-weight:600}
    .reel-video{position:relative;aspect-ratio:16/9;background:linear-gradient(150deg,#0E3A2C,#17533C);display:flex;align-items:center;justify-content:center}
    .reel-video .court{position:absolute;inset:16px;border:2px solid
```

```
rgba(199,242,176,.30);border-radius:6px}
.reel-video .court:before{content:"";position:absolute;left:50%;top:0;bottom:0;width:2px;background:rgba(199,242,176,.30)}
.reel-video .court:after{content:"";position:absolute;top:50%;left:0;right:0;height:2px;background:rgba(199,242,176,.22)}
.reel-video .play{width:60px;height:60px;border-radius:50%;background:var(--lime);display:flex;align-items:center;justify-content:center;color:#0C261F;font-size:1.4rem;z-index:2;padding-left:4px;box-shadow:0 10px 26px rgba(0,0,0,.35)}
.reel-video .badge{position:absolute;top:12px;left:12px;background:rgba(12,38,31,.72);color:var(--lime);font-size:.7rem;font-weight:700;letter-spacing:.06em;padding:.26rem .6rem;border-radius:999px;z-index:2}
.reel-video .wm{position:absolute;bottom:10px;right:12px;color:rgba(234,243,238,.78);font-size:.66rem;font-weight:500;z-index:2}
.reel-body{padding:.95rem 1.05rem 1.1rem}
.reel-body .lbl{font-size:.72rem;font-weight:600;letter-spacing:.04em;text-transform:uppercase;color:var(--mut)}
.reel-track{position:relative;height:12px;border-radius:6px;background:#E8ECE6;margin:.55rem 0 .95rem}
.reel-track span{position:absolute;top:0;bottom:0;background:var(--green);border-radius:6px}
.reel-stats{display:flex;gap:10px}
.reel-stats .m{flex:1;background:var(--paper);border-radius:10px;padding:.6rem .85rem}
.reel-stats .m b{display:block;font-size:1.25rem;color:var(--tx);font-weight:700}
.reel-stats .m em{font-size:.76rem;color:var(--mut);font-style:normal}
.ask{display:flex;align-items:center;gap:.55rem;margin:.85rem 1.05rem .2rem;padding:.6rem .7rem;border:1px solid var(--line);border-radius:12px;background:var(--paper)}
.ask-ic{color:var(--green-d);font-size:.95rem}
.ask-tx{flex:1;font-size:.88rem;color:var(--tx);font-weight:500;line-height:1.3}
.ask-go{width:26px;height:26px;border-radius:8px;background:var(--green);color:#04140D;display:flex;align-items:center;justify-content:center;font-size:.85rem;flex:none}
.lbl{display:flex;justify-content:space-between;align-items:center}
.lbl .match{color:var(--green-d);font-weight:700}
.reel-track span.off{background:#D7DDD4}
.reel-track span.on{background:var(--green)}
.result{display:flex;align-items:center;gap:.7rem;margin-top:.2rem;padding:.65rem .8rem;background:var(--ink);border-radius:12px}
.result .play{width:34px;height:34px;border-radius:50%;background:var(--lime);color:#0C261F;display:flex;align-items:center;justify-content:center;flex:none;font-size:.95rem;padding-left:2px}
.result-tx{flex:1}
.result-tx b{display:block;font-size:.84rem;font-weight:700;color:#fff}
.result-tx em{display:block;font-style:normal;font-size:.73rem;color:var(--on-dark-mut)}
.result .dl{color:var(--on-dark-mut);font-size:1.1rem}
.askc .q{font-weight:600;background:var(--paper);border:1px solid var(--line);border-radius:10px;padding:.6rem .75rem;font-size:.95rem}
.askc .q i{color:var(--green-d);font-style:normal;margin-right:.35rem}
.askc .a{display:flex;gap:.5rem;color:var(--mut);font-size:.93rem;margin-top:.75rem;padding-left:.1rem}
.askc .a i{color:var(--green-d);font-style:normal;margin-top:.1rem}
.ask-note{margin-top:1.5rem;font-size:.95rem;color:var(--mut)}

/* sections */
section{padding:72px 0}
.sec-head{max-width:60ch}
.sec-head .kicker{font-size:.82rem;font-weight:700;letter-spacing:.08em;text-transform:uppercase;color:var(--green-d)}
.sec-head h2{margin:.5rem 0 0}
.sec-head p{color:var(--mut);font-size:1.08rem;margin-top:.7rem}

.grid{display:grid;gap:18px;margin-top:2.4rem}
```

```
.g4{grid-template-columns:repeat(4,1fr)}
.g3{grid-template-columns:repeat(3,1fr)}
.g2{grid-template-columns:repeat(2,1fr)}
.card{background:var(--card);border:1px solid var(--line);border-radius:var(--r);padding:1.4rem}
.card .ic{width:42px;height:42px;border-radius:11px;display:flex;align-items:center;justify-content:center;
  background:rgba(25,179,107,.12);color:var(--green-d);font-size:1.3rem;margin-bottom:.9rem}
.card h3{font-size:1.05rem}
.card p{color:var(--mut);font-size:.96rem;margin:.4rem 0 0}

.split{display:grid;grid-template-columns:1.1fr .9fr;gap:42px;align-items:center}
.panel{background:var(--ink);color:var(--on-dark);border-radius:18px;padding:2.2rem}
.panel h2{color:#fff}
.panel p{color:var(--on-dark-mut)}
.panel .pill{display:inline-block;background:rgba(199,242,176,.14);color:var(--lime);font-size:.8rem;font-weight:600;
  padding:.3rem .7rem;border-radius:999px;margin-bottom:1rem}

.road{display:grid;grid-template-columns:repeat(5,1fr);gap:14px;margin-top:2.2rem}
.road .step{background:var(--card);border:1px solid var(--line);border-radius:var(--rs);padding:1.1rem}
.road .step .n{font-size:1.4rem;color:var(--green-d)}
.road .step h3{font-size:1rem;margin-top:.5rem}
.road .step p{font-size:.88rem;color:var(--mut);margin-top:.3rem}

.honest{background:#fff;border:1px solid var(--line);border-left:4px solid var(--green);border-radius:0 var(--rs) var(--rs) 0;padding:1.2rem 1.4rem;margin-top:1.6rem}
.honest b{color:var(--tx)}
.honest p{color:var(--mut);margin:.3rem 0 0}

.nots{list-style:none;padding:0;margin:2rem 0 0;display:grid;grid-template-columns:repeat(2,1fr);gap:12px}
.nots li{display:flex;gap:.6rem;background:var(--card);border:1px solid var(--line);border-radius:var(--rs);padding:.9rem 1.1rem;color:var(--mut);font-size:.95rem}
.nots li i{color:var(--green-d);margin-top:.15rem}

details{background:var(--card);border:1px solid var(--line);border-radius:var(--rs);padding:0 1.2rem;margin-top:10px}
details summary{cursor:pointer;font-weight:600;padding:1rem 0;list-style:none;display:flex;justify-content:space-between;align-items:center;gap:1rem}
details summary::-webkit-details-marker{display:none}
details summary::after{content:"+";color:var(--green-d);font-size:1.3rem;font-weight:400}
details[open] summary::after{content:"?"}
details p{color:var(--mut);margin:0 0 1.1rem}

.cta{background:linear-gradient(160deg,var(--ink) 0%,var(--ink2) 100%);color:var(--on-dark);border-radius:20px;padding:3rem 2.4rem;text-align:center;margin:0 0 0}
.cta h2{color:#fff;max-width:20ch;margin:0 auto}
.cta p{color:var(--on-dark-mut);max-width:46ch;margin:.8rem auto 0}
.cta .hero-cta{justify-content:center}
.af{max-width:560px;margin:2.2rem auto 0;background:var(--card);border:1px solid var(--line);border-radius:var(--r);padding:1.6rem 1.5rem;text-align:left}
.af .row{margin-bottom:1.05rem}
.af label{display:block;font-size:.9rem;font-weight:600;color:var(--tx);margin-bottom:.4rem}
.af .hint{font-weight:400;color:var(--mut);font-size:.84rem}
.af input[type=text],.af input[type=email],.af select,.af textarea{width:100%;font:inherit;font-size:.95rem;color:var(--tx);background:var(--paper);border:1px solid var(--line);border-radius:10px;padding:.6rem .75rem}
.af textarea{min-height:84px;resize:vertical}
.af input:focus,.af select:focus,.af textarea:focus{outline:none;border-
```

```
color:var(--green);box-shadow:0 0 0 3px rgba(25,179,107,.15)}
  .seg{display:flex;gap:10px}
  .seg label{flex:1;margin:0;border:1px solid var(--line);border-radius:10px;padding:.7rem .8rem;cursor:pointer;font-weight:500;text-align:center}
  .seg input{position:absolute;opacity:0}
  .seg label:hover{border-color:var(--green)}
  .seg label:has(input:checked){border-color:var(--green);background:rgba(25,179,107,.07);color:var(--green-d);font-weight:700}
  .opts{display:flex;flex-wrap:wrap;gap:8px}
  .opts label{margin:0;display:inline-flex;align-items:center;gap:.4rem;border:1px solid var(--line);border-radius:999px;padding:.4rem .8rem;font-weight:500;font-size:.88rem;cursor:pointer}
  .opts label:has(:checked){border-color:var(--green);background:rgba(25,179,107,.07);color:var(--green-d)}
  .opts input{accent-color:var(--green)}
  .af .consent{display:flex;align-items:flex-start;gap:.5rem}
  .af .consent input{margin-top:.2rem;accent-color:var(--green)}
  .af .acad{display:none}
  .af.is-academy .acad{display:block}
  .hp{position:absolute;left:-9999px;width:1px;height:1px;overflow:hidden}
  .form-done{max-width:560px;margin:2rem auto 0;text-align:center;background:rgba(25,179,107,.08);border:1px solid rgba(25,179,107,.3);color:var(--green-d);border-radius:var(--r);padding:1.4rem;font-weight:600}

  footer{border-top:1px solid var(--line);padding:42px 0;color:var(--mut);font-size:.9rem}
  .foot{display:flex;flex-wrap:wrap;justify-content:space-between;gap:1.2rem;align-items:center}
  .foot a{color:var(--green-d);text-decoration:none}
  .foot a:hover{text-decoration:underline}
  .stamp{font-size:.82rem;color:var(--mut)}

  @media (max-width:860px){
    .nav-links{display:none}
    .g4{grid-template-columns:repeat(2,1fr)}
    .g3,.g2{grid-template-columns:1fr}
    .split{grid-template-columns:1fr;gap:24px}
    .road{grid-template-columns:repeat(2,1fr)}
    .nots{grid-template-columns:1fr}
    .shuttle{opacity:.1}
    .hero-grid{grid-template-columns:1fr;gap:30px}
    .reel-card{max-width:460px}
  }
</style>
</head>
<body>

<header>
  <div class="wrap nav">
    <a class="brand" href="#top">Khel<span class="play" aria-hidden="true"></span>Sutra</a>
    <nav class="nav-links" aria-label="Primary">
      <a href="#how">How it works</a>
      <a href="#diff">Why it's hard</a>
      <a href="#roadmap">Roadmap</a>
      <a href="#faq">FAQ</a>
    </nav>
    <a class="btn btn-dark" href="#join">Request access</a>
  </div>
</header>

<a id="top"></a>
<section class="hero" style="padding:0">
  <div class="wrap hero-grid">
    <div class="hero-left">
```

```
          <span class="eyebrow">Invite-only alpha · Bengaluru</span>
          <h1>Every rally, indexed. <span class="hl">The dead time, gone.</span></h1>
          <p class="lead">KhelSutra finds every rally in your session and strips out the knock-ups, walks and gaps ? so a full match becomes a clean highlight reel you can watch and download. No editing, no scrubbing.</p>
          <div class="hero-cta">
            <a class="btn btn-primary" href="#join">Request alpha access</a>
            <a class="btn btn-ghost" href="#how">See how it works</a>
          </div>
          <p class="hero-note">A phone or GoPro on a tripod is all you need. No special hardware.</p>
          <div class="loop">
            <span class="chip"><b>1.</b> Auto-index rallies</span>
            <span class="arrow">?</span>
            <span class="chip"><b>2.</b> Dead time removed</span>
            <span class="arrow">?</span>
            <span class="chip"><b>3.</b> Download your reel</span>
          </div>
        </div>
        <div class="hero-visual">
          <div class="reel-card">
            <div class="reel-head">
              <span class="dots"><i></i><i></i><i></i></span>
              <span class="file">match_2026.mp4 · indexed</span>
            </div>
            <div class="reel-body">
              <div class="lbl"><span>Rallies detected</span><span class="match">44 found</span></div>
              <div class="reel-track">
                <span class="on" style="left:2%;width:8%"></span>
                <span class="on" style="left:13%;width:6%"></span>
                <span class="on" style="left:22%;width:12%"></span>
                <span class="on" style="left:37%;width:7%"></span>
                <span class="on" style="left:47%;width:11%"></span>
                <span class="on" style="left:61%;width:6%"></span>
                <span class="on" style="left:70%;width:13%"></span>
                <span class="on" style="left:86%;width:9%"></span>
              </div>
              <div class="result">
                <span class="play" aria-hidden="true">?</span>
                <span class="result-tx"><b>Highlight reel · 44 rallies · 18 min</b><em>dead time removed · ready to download</em></span>
                <span class="dl" aria-hidden="true">?</span>
              </div>
            </div>
          </div>
        </div>
      </div>
    </section>

    <section id="how">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">What it does today</div>
          <h2>A full match, turned into rallies you can search</h2>
          <p>Upload a whole session or match. KhelSutra does the rest ? no editing skill needed.</p>
        </div>
        <div class="grid g4">
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Every rally, indexed</h3>
            <p>It finds each rally ? start, end and length ? and turns the whole match into a searchable list. Long full-session videos are fine.</p>
          </div>
```

```
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Dead time removed</h3>
            <p>Knock-ups, walks, picking up the shuttle, the gaps between points ? stripped out. What's left is only real play.</p>
          </div>
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Process once, reuse</h3>
            <p>Come back later to watch or download your reel. Re-running a video replaces its earlier result rather than piling up copies.</p>
          </div>
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Stitched reel + summary</h3>
            <p>Your selection becomes one MP4 to download and share ? with a play-time summary and a small ?Generated by Khelsutra.guru? mark.</p>
          </div>
        </div>
      </div>
    </section>

    <section id="diff" style="padding-top:0">
      <div class="wrap">
        <div class="split">
          <div>
            <div class="kicker" style="font-size:.82rem;font-weight:700;letter-spacing:.08em;text-transform:uppercase;color:var(--green-d)">The hard problem we crack</div>
            <h2 style="margin-top:.5rem">Most academy halls run several courts at once</h2>
            <p style="color:var(--mut);font-size:1.05rem">Academies already record a lot of video ? and then almost never watch it, because cutting it into useful clips by hand is tedious. The good rallies stay buried.</p>
            <p style="color:var(--mut);font-size:1.05rem">A single recording of a busy, multi-court hall is exactly where ordinary video tools choke. Pulling the <b style="color:var(--tx)">real</b> rallies out of busy, multi-court footage is the hard problem KhelSutra is built for. It already works very well on clean, single-court recordings ? and the multi-court case is what we're sharpening now.</p>
          </div>
          <div class="panel">
            <span class="pill">Capture tip</span>
            <h2 style="font-size:1.4rem">Best results: a steady, full-court shot</h2>
            <p>Camera behind the court, slightly elevated, whole court in frame. Keep it still ? a fixed tripod beats handheld every time. Shoot landscape, record the whole session, upload as-is. KhelSutra does the cutting.</p>
          </div>
        </div>
      </div>
    </section>

    <section id="sharper" style="background:#EFF4EC">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">What makes it different</div>
          <h2>It gets sharper from real academy footage</h2>
          <p>The footage real academies contribute ? especially the busy multi-court halls ? is what makes KhelSutra better over time. Early users aren't just customers; they're co-builders.</p>
        </div>
        <div class="grid g3">
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Improves with the library</h3>
            <p>When you choose to share footage and coaches confirm what it found, that growing labelled library is what sharpens future versions ? rather than staying frozen.</p>
          </div>
```

```
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Your data stays yours</h3>
            <p>We never use your video to improve the tool without your clear, explicit opt-in. We never publish your footage. You decide what, if anything, you share.</p>
          </div>
          <div class="card">
            <div class="ic" aria-hidden="true">?</div>
            <h3>Minors are protected</h3>
            <p>Footage involving minors is never used to improve the tool and is never published ? full stop.</p>
          </div>
        </div>
        <div class="honest">
          <b>Read this honestly:</b>
          <p>Today the tool gives you rally detection, reels, and summaries. It does not retrain itself as you use it, and there's no per-person tracking. "Gets sharper over time" is the model we're building toward, powered by contributed footage ? not a switch that's already on.</p>
        </div>
      </div>
    </section>

    <section id="ask">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">Coming next to the alpha</div>
          <h2>Soon: ask for the exact reel you want</h2>
          <p>Your match is already a clean index of rallies ? so the natural next step is to let you build any reel just by describing it, in plain English. This is rolling out to alpha users.</p>
        </div>
        <div class="grid g3 ask-grid">
          <div class="card askc">
            <div class="q"><i>?</i>?Every rally over 10 seconds?</div>
            <div class="a"><i>?</i> 6 rallies · a 1:32 reel of the long, end-to-end battles</div>
          </div>
          <div class="card askc">
            <div class="q"><i>?</i>?Just the third game, longest first?</div>
            <div class="a"><i>?</i> 12 rallies, reordered by length ? that game only</div>
          </div>
          <div class="card askc">
            <div class="q"><i>?</i>?The 8 best rallies from today?</div>
            <div class="a"><i>?</i> an 8-clip highlight from your latest session</div>
          </div>
        </div>
        <p class="ask-note"><b style="color:var(--tx)">Rolling out now.</b> Targeted asks ? by rally length, count, order, or which game ? come first; shot-type and player queries follow as the deeper analysis arrives.</p>
      </div>
    </section>

    <section id="roadmap">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">Where it's going</div>
          <h2>Toward an AI assistant coach</h2>
          <p>Fed by the growing video library, KhelSutra is heading from "finds rallies" to a genuine training companion. This is the direction ? not today's buttons.</p>
        </div>
        <div class="road">
          <div class="step"><div class="n">?</div><h3>Shot detection</h3><p>Smashes, drops, net shots, serves and clears within each rally.</p></div>
          <div class="step"><div class="n">?</div><h3>Deeper analysis</h3><p>Per-rally and
```

```
per-match patterns ? what's working and what isn't.</p></div>
          <div class="step"><div class="n">?</div><h3>Per-match history</h3><p>Track progress over weeks and months, not just one clip.</p></div>
          <div class="step"><div class="n">?</div><h3>Assistant coach</h3><p>Surfaces what to work on next ? supports the real coach, never replaces.</p></div>
          <div class="step"><div class="n">?</div><h3>More sports & languages</h3><p>Table tennis next; Kannada and Hindi alongside English.</p></div>
        </div>
      </div>
    </section>

    <section id="not" style="padding-top:0">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">So expectations stay honest</div>
          <h2>What KhelSutra is not</h2>
        </div>
        <ul class="nots">
          <li><i aria-hidden="true">?</i><span>An early, invite-only trial ? useful, but still improving. You'll hit rough edges.</span></li>
          <li><i aria-hidden="true">?</i><span>Not a line-calling, scoring, or umpiring system.</span></li>
          <li><i aria-hidden="true">?</i><span>Not player tracking or identification.</span></li>
          <li><i aria-hidden="true">?</i><span>Not a replacement for a coach ? a tool that saves time and adds insight.</span></li>
          <li><i aria-hidden="true">?</i><span>Shot detection and deeper analytics are on the roadmap, not available today.</span></li>
          <li><i aria-hidden="true">?</i><span>Pricing isn't set ? today's invited use is about getting it right.</span></li>
        </ul>
      </div>
    </section>

    <section id="faq" style="background:#EFF4EC">
      <div class="wrap">
        <div class="sec-head">
          <div class="kicker">Quick answers</div>
          <h2>Common questions</h2>
        </div>
        <div style="margin-top:1.6rem">
          <details><summary>How accurate is it?</summary><p>Very good on a clean, single-court recording. A busy multi-court hall is harder ? and is exactly what we're improving. The best answer is to show you a sample reel of your own court rather than quote a number.</p></details>
          <details><summary>Can I choose which rallies go in the reel?</summary><p>Today your reel includes the rallies KhelSutra found, with the dead time removed. Choosing by length, count, order or which game ? just by asking in plain English ? is rolling out to alpha users next; shot-type and player queries follow as the deeper analysis arrives.</p></details>
          <details><summary>Do I need a special camera or hardware?</summary><p>No. A phone or GoPro on a tripod, behind the court, whole court in frame. We provide a one-page capture checklist at setup.</p></details>
          <details><summary>Is my video private? What happens to it?</summary><p>We treat your footage as private: we never publish it, and we never use it to improve the tool without your explicit opt-in. These are large videos that need real processing, so your video is handled securely on our infrastructure to make your reel. Footage involving minors is never used and never published.</p></details>
          <details><summary>Does it work for doubles and tournaments?</summary><p>It finds rallies in doubles too. Tournament footage varies ? a clean, fixed, full-court recording works well; crowded multi-court tournament halls are the hard case we're specifically working on.</p></details>
          <details><summary>Which sports?</summary><p>Badminton today. Table tennis and other racket sports are on the roadmap.</p></details>
          <details><summary>How do I join?</summary><p>It's an invite-only private trial. The best next step is to let us record a session and show you your own court and players as a
```

```
reel. Email <a href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a>.</p></details>
        </div>
      </div>
    </section>

    <section id="join" style="padding-bottom:90px">
      <div class="wrap">
        <div class="sec-head" style="text-align:center;max-width:none">
          <div class="kicker">Request access</div>
          <h2>Get into the KhelSutra alpha</h2>
          <p>Invite-only. Tell us a little about your setup ? it helps us get you the best results, and shapes what we build next.</p>
        </div>

        <form id="af" class="af" novalidate>
          <div class="row">
            <label>I'm a?</label>
            <div class="seg">
              <label><input type="radio" name="segment" value="academy" required><span>Coaching academy / club</span></label>
              <label><input type="radio" name="segment" value="individual"><span>Individual player</span></label>
            </div>
          </div>

          <div class="row">
            <label for="af-name">Name</label>
            <input id="af-name" name="name" type="text" autocomplete="name" required>
          </div>
          <div class="row">
            <label for="af-email">Email <span class="hint">(so we can reach you)</span></label>
            <input id="af-email" name="email" type="email" autocomplete="email" required>
          </div>
          <div class="row">
            <label for="af-city">City <span class="hint">(optional)</span></label>
            <input id="af-city" name="city" type="text">
          </div>

          <div class="row acad">
            <label for="af-org">Academy / club name <span class="hint">(optional)</span></label>
            <input id="af-org" name="org_name" type="text">
          </div>
          <div class="row acad">
            <label for="af-courts">How many courts do you record? <span class="hint">(optional)</span></label>
            <select id="af-courts" name="courts">
              <option value="">Select?</option>
              <option value="1">1 court</option>
              <option value="2-4">2?4 courts</option>
              <option value="5+">5+ courts</option>
            </select>
          </div>

          <div class="row">
            <label>Do you have a GPU machine that could process video locally?</label>
            <div class="opts">
              <label><input type="radio" name="has_gpu" value="yes">Yes ? a gaming / GPU PC or laptop</label>
              <label><input type="radio" name="has_gpu" value="unsure">Not sure</label>
              <label><input type="radio" name="has_gpu" value="no">No</label>
              <label><input type="radio" name="has_gpu" value="prefer-cloud">I'd prefer it runs in the cloud</label>
            </div>
```

```
          </div>

          <div class="row">
            <label>Where do your recordings live / how would you share them? <span class="hint">(pick any)</span></label>
            <div class="opts">
              <label><input type="checkbox" name="storage" value="gdrive">Google Drive</label>
              <label><input type="checkbox" name="storage" value="onedrive">OneDrive / MS 365</label>
              <label><input type="checkbox" name="storage" value="dropbox">Dropbox</label>
              <label><input type="checkbox" name="storage" value="bucket">Cloud bucket (S3/GCS)</label>
              <label><input type="checkbox" name="storage" value="local">SD card / local drive</label>
              <label><input type="checkbox" name="storage" value="unsure">Not sure</label>
            </div>
          </div>

          <div class="row">
            <label for="af-msg">Anything else? <span class="hint">(how you record, what you'd want?)</span></label>
            <textarea id="af-msg" name="message"></textarea>
          </div>

          <div class="row consent">
            <input id="af-consent" name="consent" type="checkbox" value="yes">
            <label for="af-consent" style="margin:0;font-weight:400;color:var(--mut)">Okay to email me about the KhelSutra alpha.</label>
          </div>

          <input class="hp" type="text" name="company" tabindex="-1" autocomplete="off" aria-hidden="true">

          <button class="btn btn-primary" type="submit" style="width:100%;justify-content:center">Request access</button>
          <p class="hint" style="text-align:center;margin:.85rem 0 0">We never publish your footage; footage involving minors is never used to improve the tool.</p>
        </form>

        <p id="af-done" class="form-done" hidden>Thanks ? we've got it. We'll be in touch at the email you gave. ?</p>
      </div>
    </section>

    <footer>
      <div class="wrap foot">
        <div>
          <div class="brand" style="font-size:1.05rem">Khel<span class="play" aria-hidden="true"></span>Sutra</div>
          <div class="stamp" style="margin-top:.4rem">Automatic badminton highlights · Bengaluru, India</div>
        </div>
        <div style="text-align:right">
          <div><a href="/capture.html">Capture checklist</a> · <a href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a> · <a href="https://khelsutra.guru">khelsutra.guru</a></div>
          <div class="stamp" style="margin-top:.4rem">© 2026 KhelSutra · Invite-only alpha · Reels carry a "Generated by Khelsutra.guru" mark</div>
        </div>
      </div>
    </footer>

    <!-- Cloudflare Web Analytics: paste your token from Cloudflare dashboard ? Analytics ? Web Analytics ? Add a site.
```

```
      (Or skip this snippet entirely and enable "Automatic setup" once the domain is proxied through Cloudflare.) -->
<script defer src="https://static.cloudflareinsights.com/beacon.min.js" data-cf-beacon='{"token": "REPLACE_WITH_CF_TOKEN"}'></script>

<script>
(function(){
  var f = document.getElementById('af');
  if(!f) return;
  f.addEventListener('change', function(e){
    if(e.target.name === 'segment') f.classList.toggle('is-academy', e.target.value === 'academy');
  });
  function mailtoFallback(p){
    var lines = [
      'Segment: ' + (p.segment||''),
      'Name: ' + (p.name||''),
      'Email: ' + (p.email||''),
      'City: ' + (p.city||''),
      p.segment === 'academy' ? 'Academy: ' + (p.org_name||'') + ' (' + (p.courts||'?') + ' courts)' : null,
      'GPU: ' + (p.has_gpu||''),
      'Storage: ' + ((p.storage||[]).join(', ')),
      'Message: ' + (p.message||''),
      'Consent: ' + (p.consent ? 'yes' : 'no')
    ].filter(Boolean).join('\n');
    return 'mailto:hello@khelsutra.guru?subject=' + encodeURIComponent('KhelSutra alpha access') +
           '&body=' + encodeURIComponent('Hi KhelSutra team, I would like to try the alpha.\n\n' + lines);
  }
  f.addEventListener('submit', async function(e){
    e.preventDefault();
    if(f.company && f.company.value) return;            // honeypot: silently drop bots
    if(!f.checkValidity()){ f.reportValidity(); return; }
    var fd = new FormData(f);
    var p = {
      segment: fd.get('segment'), name: fd.get('name'), email: fd.get('email'),
      city: fd.get('city'), org_name: fd.get('org_name'), courts: fd.get('courts'),
      has_gpu: fd.get('has_gpu'), storage: fd.getAll('storage'),
      message: fd.get('message'), consent: !!fd.get('consent')
    };
    var ok = false;
    try {
      var r = await fetch('/api/request-access', {method:'POST', headers:{'content-type':'application/json'}, body: JSON.stringify(p)});
      ok = r.ok;
    } catch(_){ ok = false; }
    if(ok){ f.hidden = true; document.getElementById('af-done').hidden = false; }
    else { window.location.href = mailtoFallback(p); }   // backend not live yet ? prefilled email
  });
})();
</script>

</body>
</html>

```

5. user (2026-06-20T10:03:06.072Z)

```
# Roora Annotation Guidelines ? Rally Golden Set

**Version: v8 (2026-06-13).** Supersedes the older Drive copies (v3/v5/v6/v7) **and** the internal
```

"DRAFT v2" label ? those numbers never matched each other; **v8 is the single label to use with Roora going forward.** Content is otherwise the finalized 2026-06-12 guideline plus the fusion-driven additions marked **`? ADDED (fusion)`** below.

> **?? Canonical home / provenance.** **This in-repo file is the sole canonical copy of the guideline**
> ? it lives here because the multi-signal-fusion features that motivate the new fields (court
> calibration, singles/doubles) originate in *this* repo (see
> MULTI_SIGNAL_FUSION_PLAN.md and
> GOLDEN_SET_VENDORING.md, ADR-006). The annotation deliverable still imports
> into the sibling **`sports-data-collector`** repo via `python -m src.cli.curate import-local`; if a
> mirror is ever staged there for the collector's intake docs, it must be regenerated from this file
> (and the `? ADDED (fusion)` callouts are exactly the deltas to port).
>
> **`TODO(avidullu)` markers are preserved** ? they are owner-only items (Drive links, intake
> manifest, NDA counsel review) and must be filled before sending to Roora. Search for `TODO(avidullu)`.

Brief (vendor spec) + Illustrated Guide (rater how-to), in one doc.

PART A ? Roora Annotation Brief (vendor-facing spec)

Project: racket-sport rally segmentation golden / regression dataset.
Sports in scope: Badminton, Pickleball, Table Tennis (each in singles AND doubles).

This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV that imports directly via `python -m src.cli.curate import-local`.

0. What we need in one sentence

For each provided match video, produce ONE CSV file listing EVERY rally with its start time, end time, number of shots, and how it ended ? EXCLUDING all dead time.

1. Deliverable format (canonical)

One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`). One row per rally, chronological, header row required.

Columns:

Column	Type / values	Meaning
`rally_number`	integer, 1-based, chronological ? no gaps, no repeats	rally index
`start_mmss`	`m:ss.mmm` (e.g. `1:12.40`)	rater-entered serve-contact time
`end_mmss`	`m:ss.mmm`	rater-entered ball/shuttle-dead time
`start_time`	decimal **seconds** (e.g. `72.40`) ? **AUTO-FILLED** from `start_mmss`	machine time
`end_time`	decimal **seconds** ? AUTO-FILLED from `end_mmss`	machine time
`shots_count`	integer	total racket/paddle/bat contacts (serve = shot 1)
`ending_reason`	`winner` | `forced_error` | `unforced_error` | `service_fault` | `let` | `other`	WHY it ended
`last_shot_outcome`	`in` | `net` | `out` | `n_a`	WHERE the last shot landed (`n_a` for service_fault/let)
`score_before`	optional, `"A-B"` ? blank if not confidently readable	score before

|
| `score_after` | optional, `"A-B"` | score after |
| `notes` | optional free text ? **REQUIRED when `ending_reason = other`** | edge-case note |

Times (mm:ss ? decimal seconds): raters type only `start_mmss` / `end_mmss`. The template Google Sheet fills `start_time` / `end_time` with this formula (shown for `start_time`, row 2):

```
=IF(B2="","", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 + VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
```

On "Download as CSV" the computed decimal seconds export. (If your tool already shows decimal seconds, type those into `start_time` / `end_time` and leave the mm:ss columns blank.)

Example (badminton):

```
rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot_outcome,score_before,score_after,notes
1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no pressure)
```

Critical format rules:
- `start_time` / `end_time` that reach us MUST be decimal seconds (the formula guarantees this).
- Times are relative to the **START OF THE FILE WE PROVIDE**. Do NOT trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum. (The file we share may be an **annotation-optimized copy** of the original ? lighter and faster to frame-step, but frame-for-frame identical. Treat it exactly like the original: annotate it as-is.)
- `end_time > start_time` for every row.
- No overlaps: `end_time[N] <= start_time[N+1]`.
- Everything between one rally's `end_time` and the next rally's `start_time` is DEAD TIME and gets NO row.

2. The three non-negotiable rules (full detail + pictures in the Guide)

1. **IGNORE ALL DEAD TIME.** Only label active play.
2. **INCLUDE EVERY RALLY ? EXCLUDE NONE.** Every served point, including service faults, 1?2 shot rallies, and lets/replays. A missed rally is the worst error in this project.
   - **EXCEPTION:** warm-up / knock-up is NOT a rally ? pre-match and between-games warm-up looks like rallying but isn't scored play, so don't label it. Tell it apart by: no real serve; cooperative (not competitive) hitting; the score not progressing; often several shuttles/balls in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag it.
   - **SECOND EXCEPTION:** a rally already in progress when the file starts (or still in progress when it ends) gets NO row ? its true start (or end) is outside the footage and must never be guessed or extrapolated. Note it in the delivery instead.
3. **A SHOT = ONE CONTACT** with the racket/paddle/bat. Serve = shot 1. Bounces and net-clips are not shots.

3. Scope & phasing

- **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table tennis), each
  < 3 min with ~10?12 rallies. This is the paid calibration round; we review jointly and lock the
  Guide before scale-up.
- **Phase B ? Scale (after a successful pilot):** ~12?15 videos PER SPORT (~36?45 total), each ~30 min.
  Same format, rules, and template.

Doubles are included across the program. Rally rules are identical to singles; the badminton- and
pickleball-specific doubles notes are in the Guide.

4. Quality bar (acceptance criteria)

Automated validation (no overlaps, positive durations, schema-valid) runs on every file, then a ~20%
human spot-check. A file is returned for rework (in scope) if it misses:

- **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
- **Boundaries:** `start_time` / `end_time` within **±0.3 s** of true serve-contact / landing.
  - ?? Boundaries that all sit on a frame-sampling grid (e.g. every 30th frame) STRUCTURALLY FAIL this
    bar and are detected automatically ? the whole file is returned for rework. See §7 for the
    required method.
- **`shots_count`:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
- **`ending_reason`:** one of the six allowed values, per the Guide's decision tree.
- **Format:** decimal seconds present, valid columns, non-overlapping, chronological.

> **? ADDED (fusion) ? court calibration as a day-1 quality lever.** The owner's pipeline produced a
> bad highlight where a held shuttle was mislabeled as a rally. The boundary definition (§ Part 3:
> start = serve **contact**, not the toss/hold) already excludes that ? so faithful boundaries are
> themselves a quality checkpoint. The one NEW field we need (next section) is **court calibration**,
> which lets the detector reason about court occupancy and net-crossings.

A. Court calibration & intake manifest   ? ADDED (fusion)

> **? ADDED (fusion).** *Why:* the multi-signal detector
> (MULTI_SIGNAL_FUSION_PLAN.md) needs to know **where the court is** and
> **where the net is** to (a) ignore people outside the court, (b) require ?2 players on court for a
> rally, and (c) count shuttle net-crossings. This is a **one-time, per-video** annotation ? cheap,
> high-leverage, and **required in deliverables from day 1.**

Per-video, recorded once in the intake manifest (NOT in the per-rally CSV):

Manifest column	Type	Meaning
`video_id`	string	(existing) we supply it
`fps`	number	(existing) we supply it
`court_corners`	4 points `(x,y)` in pixels, clockwise from the near-left corner	the playable court rectangle
`net_line`	2 endpoints `(x,y)` in pixels	the net, left post ? right post
`singles_or_doubles`	`singles` | `doubles`	sets the expected on-court player count (2 vs 4)

- Pick **one clear frame** with the empty/visible court and click the 4 corners + 2 net endpoints.
- If the camera does not move for the whole video (the normal case), one calibration covers the file.
  If the camera is moved mid-file, flag it ? we'll split the file.
- Worked example (badminton, 1920×1080): `court_corners=[(360,980),(1560,980),(1330,300),(590,300)]`,
  `net_line=[(470,640),(1450,640)]`, `singles_or_doubles=singles`.

(After the first batch we may add more per-shot fields ? see the §B appendix ? and renegotiate price.)

5. Per-sport quick reference (full version + examples in the Guide)

- **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle floors /
  fault · shot: each racket?shuttle contact.
- **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double bounce / out /
  net / fault · shot: each paddle?ball contact (floor bounces are NOT shots; two-bounce rule is normal play).
- **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal return · shot:
  each racket?ball contact (table bounces are NOT shots). Net-cord serve = let.

6. `ending_reason` values (decision tree in the Guide)

This vocabulary attributes WHY the point ended; it is shared across all net-separated sports
(badminton, pickleball, table tennis, tennis). Record WHERE the last shot physically landed in the
separate `last_shot_outcome` column (`in` | `net` | `out` | `n_a`) ? e.g. a forced error that flew
long is `ending_reason=forced_error, last_shot_outcome=out`.

- `winner` ? last shot landed IN and the opponent made no legal contact (a clean winner).
- `forced_error` ? the losing player erred (netted / out / missed) UNDER PRESSURE ? stretched,
  wrong-footed, or rushed by the opponent's strong shot.
- `unforced_error` ? the losing player erred on a ROUTINE, UNPRESSURED ball they had time and position
  to make.
- `service_fault` ? the serve itself failed / was illegal.
- `let` ? the point was replayed.
- `other` ? anything else / cannot tell. MUST add a note.

7. Capture method (required for boundary frames)

Use any tool that shows current time (`mm:ss.mmm` or seconds, or frame + known fps) and can step one
frame forward/back: a frame-steppable player (VLC frame-step) or a timeline tool (Label Studio /
CVAT). The deliverable is the CSV in the schema above ? or, if you work in CVAT, the CVAT export as-is
(we recompute each rally's time from its box frame range; seconds = `frame_number / fps`, fps provided
per video).

Boundary placement is the non-negotiable part:
- Set up the CVAT task at **FRAME STEP 1** on the file we provide (it is encoded to make this fast ?
  frame-stepping and back-scrubbing stay snappy).
- The FIRST box of every rally goes on the EXACT serve-contact frame; the LAST box on

the EXACT
  shuttle/ball-dead frame. Frame-step to find them.
- Boxes IN BETWEEN may stay sparse (e.g. every 30th frame) ? only the two boundary frames must be exact.
- Self-check before submitting: every start/end within ±0.3 s of the true frame (Guide).

8. Logistics & contacts

- **TODO(avidullu):** paste the Google Drive folder link where source videos are shared with Roora.
- Per-video fps + `video_id` (+ the calibration columns from §A): provided in the intake manifest
  (template `INTAKE_MANIFEST_TEMPLATE.csv`). **TODO(avidullu):** fill the manifest (one row per video)
  and share its link.
- **TODO(avidullu):** paste the Drive folder link where Roora uploads completed `<video_id>.csv` files.
- Blank annotation template Sheet (mm:ss ? seconds formula from §1 pre-filled):
  TODO(avidullu): create it and paste the link.
- **TODO(avidullu):** add the escalation channel (email / WhatsApp group) for edge-case questions.
- NDA before any footage is shared ? template `NDA_TEMPLATE.md`. **TODO(avidullu):** have counsel
  review, fill the blanks, and have Roora sign.
- Roora contact on file: info@rooragrp.com (HSR Layout, Bangalore).

> **? ADDED (fusion) ? data-handling guardrails (put in the brief / contract):**
> - **Exclude minors entirely.** If footage contains children, that video does **not** enter the
>   labeled pool ? do not annotate it; flag it back to us.
> - **No biometric / identity labeling.** Refer to players only by **per-video pseudonyms** (e.g.
>   "near side" / "far side") ? never a real name, and never a cross-video person ID. We do **not**
>   collect gait or biometric annotations.
> - **No pose / skeleton / keypoint annotation** is requested.
> - **Work-for-hire IP assignment** (atop the NDA): the labels Roora produces are assigned to us as
>   owned work product.

9. Pricing request (please quote)

1. Price per rally and per minute of footage.
2. Pilot price (3 short videos, ~30?36 rallies) as a paid calibration round.
3. Indicative scale price for ~36?45 longer videos.
4. Turnaround for the pilot and throughput (rallies/day) at scale.
5. QA process and rework policy.
6. Whether annotators have racket-sport familiarity (helpful, not required).
7. Confirmation you can deliver the CSV schema above **(incl. the §A intake-manifest calibration columns).**

PART B ? Illustrated Annotation Guide (rater how-to)

Sports: Badminton, Pickleball, Table Tennis (singles AND doubles).

> **Illustrations:** boxes marked `[SCREENSHOT: ...]` are placeholders.
> **TODO(avidullu):** replace every `[SCREENSHOT: ...]` placeholder with a real frame grab from the
> pilot videos before sending this guide to Roora.

Part 1 ? What you are producing

One CSV per video (from the template Sheet), one row per rally:

`rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes`.

You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`, `last_shot_outcome` (+ optional score/notes). The `start_time` / `end_time` decimal-second columns FILL THEMSELVES via a formula already in the template.

For every rally you record:
- When it started (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
- When it ended (shuttle/ball dead) ? `end_mmss`
- How many shots happened ? `shots_count`
- Why it ended ? `ending_reason` · Where the last shot landed ? `last_shot_outcome`

Everything else (gaps between rallies) you IGNORE.

Part 2 ? The three golden rules

RULE 1: IGNORE ALL DEAD TIME. A rally is CONTINUOUS active play. The moment play stops, the rally is over. The time until the next serve is DEAD TIME and is never labeled.

```
  DEAD        RALLY 1                 DEAD             RALLY 2            DEAD
 (ignore) |==================|     (ignore)     |==============|     (ignore)
          ^                  ^                  ^              ^
       serve              shuttle            serve         shuttle
       contact            lands            contact        lands
       =start_1           =end_1           =start_2        =end_2
```

Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat, bouncing the ball before serving, the score being announced, replays/slow-mo, crowd or coach shots, changeovers, towel/medical timeouts, and PRE-MATCH / BETWEEN-GAMES WARM-UP (knock-up) ? see the Rule 2 caveat below.

RULE 2: INCLUDE EVERY RALLY ? EVEN THE UGLY ONES. Every served point gets a row, including:
- Service faults (serve into net/out/illegal) ? usually `shots_count = 1`, `ending_reason = service_fault`.
- Very short rallies (serve + one error).
- Lets / replays ? `ending_reason = let`; the replayed point is its OWN next row.

A missed rally is the worst error in this project ? the model must learn that short and failed rallies are still rallies.

> ?? **BUT WARM-UP / KNOCK-UP IS NOT A RALLY** ? a common trap, because Rule 2 says "include
> everything." Pre-match and between-games warm-up LOOKS like rallying but is not scored play; do not
> label it. How to tell it apart:
> - **No real serve.** Real play starts with a proper serve from the service position (diagonal,
>   behind the service line). Warm-up has players feeding cooperatively from mid-court with no serve.
> - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is trying to win the
>   point (no smashes-to-win, no scrambling/diving).
> - **Score isn't progressing.** No umpire / scoreboard not running / players casually retrieve and
>   re-feed; the score doesn't change. Often several shuttles/balls in use at once.

> - **Per sport:** badminton & pickleball ? real play starts with the underhand serve from the service
>   court / behind the baseline; table tennis ? real play starts with the tossed, two-bounce serve.
>
> When unsure: find the first proper serve where the score starts to progress ? labeling begins there.
> If you genuinely can't tell whether the match has started, flag the clip to us ? don't guess.

RULE 3: A SHOT = ONE CONTACT WITH THE RACKET / PADDLE / BAT. Count every time the shuttle/ball
touches a racket, paddle, or bat. The serve is shot 1.
- Count: any racket/paddle/bat contact (serve, return, smash, block?).
- Don't count: floor bounces, table bounces, net clips, the toss before a serve.

5-shot example: serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5) ? `shots_count = 5`,
`ending_reason = winner`.

Part 3 ? Where exactly does a rally start and end?

START (`start_mmss`): the exact frame the server's racket/paddle/bat CONTACTS the shuttle/ball.
Not the toss, not the wind-up ? the contact frame. `[SCREENSHOT: serve-contact frame]`

> **? NOTE (fusion):** this is the field that fixes the "held-shuttle" false positive ? a player
> holding/flicking the shuttle before serving is **dead time**, so it never gets a row. Faithful
> serve-**contact** boundaries are exactly what the detector is graded against.

END (`end_mmss`): the exact frame the shuttle/ball becomes DEAD:
- Badminton: shuttle touches the floor, or play stops on a fault.
- Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
- Table tennis: the second bounce on one side, a non-serve net hit, or off the table.

`[SCREENSHOT: end frame]`

If occluded/off-screen, pick the closest clear frame and add a note. **Boundary tolerance: ±0.3 s
(~±9 frames at 30 fps).**

Part 4 ? `ending_reason`: how to choose

Pick exactly ONE value for how the rally ended. Stop at the first match:
1. Did the SERVE itself fail / was illegal (and ended the point)? ? `service_fault`
2. Was the point REPLAYED (let / interruption)? ? `let`
3. Did the final shot land IN and the opponent make NO legal contact? ? `winner` (a clean winner)
4. Did the losing player make an error (netted / out / missed) UNDER PRESSURE ? stretched,
   wrong-footed, or rushed by a strong shot? ? `forced_error`
5. Did the losing player make an error on a ROUTINE, UNPRESSURED ball they had the time and position
   to make? ? `unforced_error`
6. None of the above / cannot tell ? `other` (add a note)

FORCED vs UNFORCED ? the one judgment call. Decide using the opponent's PRECEDING shot:
- `forced_error` = the opponent's shot CAUSED the miss (pace, placement, deception; the player barely
  reached it or was pulled out of position).
- `unforced_error` = a clean, makeable ball the player missed on their own.
- When genuinely unsure between the two, choose `unforced_error` and add a brief note.

Notes:
- A clean winner means NO legal contact by the opponent. If they got a racket/paddle/bat on it but
  netted or hit out, that is a `forced_error` / `unforced_error` (their shot), not a winner.
- A serve into the net is `service_fault` (rule 1 beats everything).

THEN SET `last_shot_outcome` ? where the last shot physically went. This is a SEPARATE,
observational column (no judgment): just look at the last shot.
- `in` ? landed in (winners; also a put-away the opponent couldn't reach).
- `net` ? went into the net.
- `out` ? landed outside the court / past the lines.
- `n_a` ? not applicable: use for `service_fault` and `let`.

So each rally gets both: WHY (`ending_reason`) and WHERE (`last_shot_outcome`) ? e.g. a smash winner =
`winner, in`; a clear forced long = `forced_error, out`; a routine ball netted = `unforced_error, net`.

Part 5 ? Per-sport detail

Badminton
- **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
- **Rally end:** shuttle hits the floor, or play stops on a fault.
- **Shots:** every racket?shuttle contact; serve = shot 1.
- **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside service court,
  contact above waist, foot fault.

Worked example: `[SCREENSHOT: badminton serve]`
- Rally 4: serve `1:28.30`, long rally, smash winner, shuttle lands `1:44.10`, 12 contacts.
  `4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, in, 3-5, 4-5,`
- Rally 5: serve `1:57.00` straight into the net.
  `5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, n_a, 4-5, 4-6,`

Badminton ? doubles (2 per side). All rally rules are IDENTICAL to singles:
- start = serve contact; end = shuttle down / fault; a SHOT = any racket?shuttle contact by EITHER
  player on a side.
- `shots_count` = TOTAL contacts across both teams in the rally. Do not track which player hit ? just
  count every contact.
- Net exchanges (drives, flat pushes) come fast ? frame-step so you neither miss nor double-count.
- Only the designated receiver returns the serve; if the wrong player returns, the rally ends on a
  fault ? still a row (`service_fault` if it's a serve/receiving-position fault, else `other` + note).
- Two teammates may NOT both hit the shuttle in succession (double-hit). Two contacts on the SAME side
  back-to-back = a fault ending the rally ? count both contacts, then the rally ends.

Pickleball
- **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley zone (the
  "kitchen"). Start = paddle?ball contact.
- **Two-bounce rule:** the serve must bounce once in the receiver's court and the return must bounce
  too, before either side may volley. These bounces are NORMAL play, not rally ends, and bounces are
  NOT shots.
- **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a

fault (e.g. volleying in the kitchen).
- **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not shots.
- **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen, foot fault.

Worked example: `[SCREENSHOT: pickleball serve + kitchen line]`
- Rally 2: serve `0:33.10`; serve bounces, return bounces, a few volleys; receiver nets an easy one at `0:41.85`; 6 paddle contacts.
  `2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, net, 1-0, 2-0, easy ball netted`

Pickleball ? doubles (the standard format, 2 per side). All rally rules IDENTICAL to singles:
- start = serve contact; a SHOT = any paddle?ball contact by EITHER teammate; floor bounces still not shots; two-bounce rule still applies.
- `shots_count` = TOTAL paddle contacts across both teams; don't track which player.
- Serving sequence (FYI only ? you do NOT annotate it): both partners serve before the serve passes to the opponents, except the very first service turn of the game. The called score has three numbers. Ignore for annotation; just mark boundaries / shots / ending.
- If both teammates contact the ball in succession, that's a fault ? rally ends; count the contacts, set the ending per the fault.

Table tennis
- **Serve:** ball tossed up from an open palm, then struck so it bounces once on each side. Start = the racket?ball strike (not the toss).
- **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes off the table, or is hit into the net on a non-serve shot.
- **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not shots.
- **Let:** a serve that clips the net but is otherwise legal ? the point is replayed. `ending_reason = let`, and the replay is its own next row.
- **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off the table.
- **Speed warning:** contacts can be < 0.2 s apart ? step frame-by-frame to count shots.
- **Doubles** (if a clip appears): partners must hit in STRICT ALTERNATION and serve diagonally; a missed alternation is a fault that ends the rally. Same rule ? count every racket?ball contact as a shot; `shots_count` is the team total.

Worked example: `[SCREENSHOT: table-tennis serve contact]`
- Rally 9: serve `4:10.40`; fast 7-contact rally; forehand winner, opponent no contact; dead `4:14.05`.
  `9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, in, 6-9, 6-10,`
- Rally 10: serve `4:22.10` clips the net (otherwise good) ? let.
  `10, 4:22.10, 4:23.00, (auto), (auto), 1, let, n_a, 6-10, 6-10, net-cord serve, replayed`

Part 6 ? Edge cases & FAQ

- **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`, never overlap.
- **Interrupted & replayed?** Label the interrupted attempt as its own row (`ending_reason=let`), then the replayed point as the next row.
- **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start

at the actual serve contact; don't label the aborted toss.
- **Opponent reached the ball but netted/hit out?** That's a `forced_error`/`unforced_error` (their shot), not winner ? forced if your shot pressured them, unforced if it was routine.
- **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
- **"Let" where the rules say play on?** If play continued, it's NOT a let ? keep one rally and judge the real ending.
- **Warm-up / knock-up / practice points?** Not real rallies ? do not label them. Start at the first real, scored serve. See the Rule 2 caveat (Part 2). When unsure whether the match has started, flag the clip ? don't guess.
- **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player hits it. Never attribute `shots_count` to an individual, just total the contacts.
- **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count = 0`, `ending_reason = service_fault`, add a note. A graze = 1 contact = 1 shot.

Part 7 ? Self-check before you submit (per video)

- [ ] `rally_number` is 1,2,3? no gaps or repeats.
- [ ] Every `start_time < end_time` (check the auto-filled decimal columns populated).
- [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
- [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look right.
- [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
- [ ] Every served point present, including service faults and lets.
- [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).
- [ ] `ending_reason` in {winner, forced_error, unforced_error, service_fault, let, other}; notes filled when `other`.
- [ ] `last_shot_outcome` in {in, net, out, n_a} (`n_a` for service_fault/let).
- [ ] File downloaded as `<video_id>.csv`, header row present, UTF-8.
- [ ] Did NOT trim / re-encode / rename the source video.
- [ ] **? (fusion)** intake-manifest row filled: `court_corners`, `net_line`, `singles_or_doubles`.

Part 8 ? Mini glossary

- **Rally** ? one continuous passage of play from serve contact to the shuttle/ball dead.
- **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either teammate in doubles).
- **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
- **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
- **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
- **Winner / forced_error / unforced_error** ? the final shot was a clean winner (opponent made no legal contact) / the loser's error was forced by a pressuring shot / the loser's error was on a routine, unpressured ball.
- **start_time / end_time** ? decimal seconds from the start of the provided file (auto-filled from mm:ss).
- **? court_corners / net_line / singles_or_doubles** ? per-video calibration in the intake manifest (§A).

Appendix B ? Tier-B extensions (NOT in the day-1 deliverable)   ? ADDED (fusion)

> Pilot these on the 3-video calibration round, then renegotiate price before scaling. **Do not start

```
> these without explicit instruction** ? they are listed so Roora can quote them.
>
> - **Per-shot hit timestamps** ? a timestamp for *each* racket/paddle/bat contact (not just the
>   `shots_count` total). Extends the §7 CVAT method (box every contact, not only the two boundary
>   frames). Trains/validates the audio hit-detector and shot-cadence features.
> - **Per-shot type** ? for each contact, a label from {serve, clear, drop, smash, net, drive, lift}.
>   Needs its own short guideline + an inter-annotator agreement check.

```

6. assistant (2026-06-20T10:03:09.581Z)

```
Now let me look at the rally-annotator repo to understand what's actually real today, since the
USP hinges on it. Let me also pull the repos fresh per the convention.
```

7. user (2026-06-20T10:03:12.363Z)

```
Already up to date.
---BRANCH---
main
---REMOTE---
origin  https://github.com/avidullu/rally-annotator.git (fetch)
origin  https://github.com/avidullu/rally-annotator.git (push)
```

8. user (2026-06-20T10:03:12.448Z)

```
C:\Users\avidu\Projects\rally-annotator\LICENSE
C:\Users\avidu\Projects\rally-annotator\.gitignore
C:\Users\avidu\Projects\rally-annotator\.git\description
C:\Users\avidu\Projects\rally-annotator\.git\hooks\applypatch-msg.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\commit-msg.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\fsmonitor-watchman.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\post-update.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-applypatch.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-commit.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\rally-annotator\.git\hooks\update.sample
C:\Users\avidu\Projects\rally-annotator\.git\info\exclude
C:\Users\avidu\Projects\rally-annotator\.git\objects\06\015abe347817c322586dddea658a8382d9b8d7
C:\Users\avidu\Projects\rally-annotator\.git\objects\36\6885e442e24c433b9511797e62bc5330885f6c
C:\Users\avidu\Projects\rally-annotator\.git\objects\3f\232efa99468a4af899f0ee46e3fc5a78316af0
C:\Users\avidu\Projects\rally-annotator\.git\objects\d6\d193674a3dfb0c58280f295d946c72c5f9d232
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\bf09962ccf48626095c050d6135e99918a38e1
C:\Users\avidu\Projects\rally-annotator\.git\objects\e1\4e44f360577f5f83c4f3cd991921f63ba3464d
C:\Users\avidu\Projects\rally-annotator\.git\objects\f9\a87512639a022203551ade1537a6f4fbf2df6e
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\2a55b1471eb204fdedb4b04a26c1753fdb4807
C:\Users\avidu\Projects\rally-annotator\.git\objects\50\7909c1a94cec6261099991be5a63b7e9e0f6a6
C:\Users\avidu\Projects\rally-annotator\.git\objects\7c\10778e78cae28070295d2c612c332f6e67e0e2
C:\Users\avidu\Projects\rally-annotator\.git\objects\8b\60c85dc70a2b8ae5c3f6cb8891402102c7b0f6
C:\Users\avidu\Projects\rally-annotator\.git\objects\b9\63d8b87bf811cfb86d661253acc41f92429462
C:\Users\avidu\Projects\rally-annotator\.git\objects\e9\c4c0f330e832b8ba687ad4c77bee0c9fcaad1f
C:\Users\avidu\Projects\rally-annotator\.git\objects\b7\84ea474c54cf9be75901b06e8b6882e3b0fff2
C:\Users\avidu\Projects\rally-annotator\.git\objects\64\09f2ed9854e9341a826038b65f79fd8a7e58d1
C:\Users\avidu\Projects\rally-annotator\.git\objects\98\d2e7d1815bb210b52e79f9df3d04f8d81ef800
C:\Users\avidu\Projects\rally-annotator\.git\objects\b0\58262152176be494a72dfd55f297376648a50c
C:\Users\avidu\Projects\rally-annotator\.git\objects\d4\c682a8ff2e3271aa60eab596be23279c59f9ae
```

```
C:\Users\avidu\Projects\rally-annotator\.git\objects\11\4489abd4a52d837b3067e4d1c54fc0bfa540a1
C:\Users\avidu\Projects\rally-annotator\.git\objects\76\31908f23c077c9420450c0c7267fa30b12507a
C:\Users\avidu\Projects\rally-annotator\.git\objects\30\26bb485e8988caf8ef01159ab3aa4458c06cd5
C:\Users\avidu\Projects\rally-annotator\.git\objects\8d\42a714e8e1f61f68964b29e669a4d5db03ff6f
C:\Users\avidu\Projects\rally-annotator\.git\objects\b6\6154751399ab39937e1761192002da2c7acfb9
C:\Users\avidu\Projects\rally-annotator\.git\objects\20\f14f77a411e808112e3eaf49282707e89c4c69
C:\Users\avidu\Projects\rally-annotator\.git\objects\d2\8c7565a6ce65007b354a573a61d25f5abea149
C:\Users\avidu\Projects\rally-annotator\.git\objects\84\e7be4c5451f9772b643ebdcf82a70a11d5c1e7
C:\Users\avidu\Projects\rally-annotator\.git\objects\37\463531b4c563410fb5a55ebe9338da2c3c98e8
C:\Users\avidu\Projects\rally-annotator\.git\objects\8c\6a969230193a68d384afcebcbada5048903f95
C:\Users\avidu\Projects\rally-annotator\.git\objects\cd\bc09d8440d69192c0c316c5521d3829d75084a
C:\Users\avidu\Projects\rally-annotator\.git\objects\d1\9980e1fd79dfd2f5e90a2038de83a5471eea0d
C:\Users\avidu\Projects\rally-annotator\.git\objects\42\4eb3ccd5f7bf916f0c912c9595387d9123c92e
C:\Users\avidu\Projects\rally-annotator\.git\objects\60\5bcf6e8ca3c69a9bc61e545a4ce6544008ffc5
C:\Users\avidu\Projects\rally-annotator\.git\objects\b9\77e2d38594c9d5ac50138fe60b9cd9c9e54f0e
C:\Users\avidu\Projects\rally-annotator\.git\objects\10\af409917d144d4c5a29f85400ba5d2da29190f
C:\Users\avidu\Projects\rally-annotator\.git\objects\37\a0ec10c6048354d3a0c824ddbbb81caa907f00
C:\Users\avidu\Projects\rally-annotator\docs\ENDING_REASONS.md
C:\Users\avidu\Projects\rally-annotator\.git\objects\84\c8a8b28b96c2cdedc1fc6378feab9a30313eb8
C:\Users\avidu\Projects\rally-annotator\.git\objects\08\1960fe673318fdb66b340a992f52d718f3f68b
C:\Users\avidu\Projects\rally-annotator\.git\objects\b0\6094ad44ec13677f61429efc19eb943f6c7331
C:\Users\avidu\Projects\rally-annotator\.git\objects\25\298d09c5d34e1058915e6add080fa2a626b0ce
C:\Users\avidu\Projects\rally-annotator\.git\objects\10\7e60616d2679a86ebf33046f1ada31fd3513c7
C:\Users\avidu\Projects\rally-annotator\.git\objects\f2\c743e07bee85896646fbea318488a813201824
C:\Users\avidu\Projects\rally-annotator\.git\objects\36\e84f1a8bbfab167cb92136802217da9668c0fc
C:\Users\avidu\Projects\rally-annotator\.git\objects\38\ef62b7938e0a7ca288e31c20070a233e0e6f8e
C:\Users\avidu\Projects\rally-annotator\.git\objects\fa\7cfffa190358fd587fa1c0d1740fe99ea27d2e
C:\Users\avidu\Projects\rally-annotator\.git\objects\9b\ffa88224f7f90e91ddf2a0b3392a10d8218d45
C:\Users\avidu\Projects\rally-annotator\.git\objects\74\f0fb41e743fc0c7df80c88b63db1c6f3f580dc
C:\Users\avidu\Projects\rally-annotator\.git\objects\d8\fc66fa92776c30bb95b8ef9a1c8e3fcc211802
C:\Users\avidu\Projects\rally-annotator\.git\objects\e1\0e5438d6c4f261119e85ed450ea8486df29504
C:\Users\avidu\Projects\rally-annotator\.git\objects\3d\5eeb4ff53eb5793c409292f3f6cf171bb3c890
C:\Users\avidu\Projects\rally-annotator\.git\objects\b5\c51b4ef23ed3e0398db8b6633ec5b1731fea9c
C:\Users\avidu\Projects\rally-annotator\.git\objects\02\7cb297e488537cf0c605251e539704bf757ef2
C:\Users\avidu\Projects\rally-annotator\.git\objects\79\bd243ab5aab1cce6a440e1a1c517796d3093e3
C:\Users\avidu\Projects\rally-annotator\.git\objects\aa\b3b7650c31aa10c03f1868d66d8564853d2f99
C:\Users\avidu\Projects\rally-annotator\.git\objects\db\d570dc2b7a5ed387ada9e05a3e79bd997b8410
C:\Users\avidu\Projects\rally-annotator\.git\objects\7b\200d4b2e9eb0073ba2cd5c04b4483c12f50382
C:\Users\avidu\Projects\rally-annotator\.git\objects\f9\e34e78824ba67b8ab071666373da5d0035728b
C:\Users\avidu\Projects\rally-annotator\.git\objects\23\ab53cae71147151e67bfa39bd809b44207266b
C:\Users\avidu\Projects\rally-annotator\.git\objects\d3\65b14261f8979c8f2c364efebfe35ed400792d
C:\Users\avidu\Projects\rally-annotator\.git\objects\67\dd08cd318d8215a76546cff5362b819de5b724
C:\Users\avidu\Projects\rally-annotator\.git\objects\9f\1f267890316f6d01f44dc5c7be6c565a3ef4a6
C:\Users\avidu\Projects\rally-annotator\.git\objects\d8\39ad74b2a0ecc98ce9cfe11c1e6a380fff64eb
C:\Users\avidu\Projects\rally-annotator\.git\objects\dd\1c54a5651f3b1acd1be67b4fbd9e248f34576a
C:\Users\avidu\Projects\rally-annotator\.git\objects\4a\92bc6a868619892384f52b083676c9770c681e
C:\Users\avidu\Projects\rally-annotator\.git\objects\df\743c7109ff5c93a39500dbd307bdd1ee97bfb1
C:\Users\avidu\Projects\rally-annotator\.git\objects\ee\86d7d372127fb5b6672931734e3f0c7fea9fb8
C:\Users\avidu\Projects\rally-annotator\.git\objects\ca\ad10fa86b0fe19f3b27538b4a092c2c2e84bf8
C:\Users\avidu\Projects\rally-annotator\.git\objects\5b\0eb2157919533da0eff733c1f8c42601d2ab49
C:\Users\avidu\Projects\rally-annotator\.git\objects\5b\f046f6c4b514935e5906f4b6b9d1a16bf52e98
C:\Users\avidu\Projects\rally-annotator\.git\objects\f2\8735847a915c9efe53b63ec6670b203b637d7e
C:\Users\avidu\Projects\rally-annotator\.git\objects\43\59082343da2a7aac90e3e1ac4e18351d3c4fac
C:\Users\avidu\Projects\rally-annotator\.git\objects\64\2d00619e1e71bed53a7b259eca3af2e189faef
C:\Users\avidu\Projects\rally-annotator\.git\objects\8d\69c13045a99ff9edac22b7dc04c8e4294a0dbd
C:\Users\avidu\Projects\rally-annotator\.git\objects\1c\dfbe89a10848abccf3fc76325bc8aaa193d26b
C:\Users\avidu\Projects\rally-annotator\.git\objects\80\04a236152999ad702fb6957c1b4c0e71d76966
C:\Users\avidu\Projects\rally-annotator\.git\objects\4f\d96d2d38a61f603ef6f417007c4ac951671632
C:\Users\avidu\Projects\rally-annotator\.git\objects\cb\5ed55afbf7ef35321e724da30655b09911a573
C:\Users\avidu\Projects\rally-annotator\.git\objects\f7\3abbbe60c437b82f34a2d690b881e4b1011191
C:\Users\avidu\Projects\rally-annotator\.git\objects\d9\c98f9ca5def221b71a02de79bb17b5c37530ee
C:\Users\avidu\Projects\rally-annotator\.git\objects\7d\427ba386c4fd6af0641ea2104a6e86f93a4910
C:\Users\avidu\Projects\rally-annotator\.git\objects\43\5267c08851c082ce142104fa0ede630a452d29
C:\Users\avidu\Projects\rally-annotator\.git\objects\9d\d141346241947603c278c34fc954f5480a782b
(Results are truncated. Consider using a more specific path or pattern.)
```

9. assistant (2026-06-20T10:03:16.572Z)

Let me list the actual tracked files in the annotator repo (excluding .git) to understand what's real.

10. user (2026-06-20T10:03:17.180Z)

```
.gitattributes
.github/workflows/ci.yml
.gitignore
CHANGELOG.md
LICENSE
README.md
docs/CSV_FORMAT.md
docs/ENDING_REASONS.md
test/README.md
test/dialog_layout.snapshot
test/dialog_test.lua
vlc/rally_annotator.lua
```

11. assistant (2026-06-20T10:03:24.018Z)

This is a VLC Lua extension. Let me read the key files: README, CSV_FORMAT, LICENSE, and the main Lua file to understand exactly what the tool does today.

12. user (2026-06-20T10:03:24.437Z)

````
# Rally Annotator

[![tests](https://github.com/avidullu/rally-annotator/actions/workflows/ci.yml/badge.svg)](https://github.com/avidullu/rally-annotator/actions/workflows/ci.yml)

A tiny **VLC plugin** to hand-label rally **start/end + point-stop reason** while you watch a match,
for **net-separated racquet sports** ? badminton, tennis, table tennis, pickleball, padel.

You watch the video in VLC, pause/scrub freely, and click **Mark START** / **Mark END** (with a
reason). It writes one CSV row per rally ? clean ground-truth labels for training/evaluating rally
(point) segmentation models, or just for cutting highlights.

```
rally_number,start_time,end_time,ending_reason,sport,shots_count
1,8.800,11.500,winner,badminton,9
2,24.389,46.589,unforced_error,badminton,21
```

Times are **decimal seconds**; `shots_count` is an **optional** rally shot/stroke count (blank if you skip it).
See [docs/CSV_FORMAT.md](docs/CSV_FORMAT.md).

## Why
Labeling rally boundaries is the slow, expensive prerequisite for any rally-detection model. Most tools
make you transcribe timestamps by hand. This lets a rater do it **inside the player they already use**,
pausing to the exact frame before each mark ? turning "watch a match" into "produce golden labels."

## Install
1. Copy `vlc/rally_annotator.lua` into your VLC Lua **extensions** folder:
````

   - **Windows:** `%APPDATA%\vlc\lua\extensions\`
   - **macOS:** `~/Library/Application Support/org.videolan.vlc/lua/extensions/`
   - **Linux:** `~/.local/share/vlc/lua/extensions/`
   (create the `extensions` folder if it doesn't exist)
2. In VLC: **Tools ? Plugins and extensions ? Reload extensions** (or just restart VLC).
3. Enable it from the **View** menu ? **Rally Annotator**. A small dialog opens and stays open while you watch.

Requires **VLC 3.0.x**. (VLC 4.0 changed the Lua input/listener API; targeting 3.x for now.)

Use
Playback is built in ? the **Back 5s · Play / Pause · Fwd 5s** row drives the VLC player from the
annotation window, so you can pause, label, and resume without ever switching to the main VLC window. **Play / Pause**
is a single toggle (it resumes when paused, pauses when playing).

1. Pick the **Sport** (it stays set across rallies).
2. When a rally begins, click **Mark START** ? it snapshots the current playback time into the **Start** field
   (pause/scrub first for frame accuracy; you can also edit the field by hand).
3. When the rally ends, click **Mark END** (fills the **End** field). This *arms* the rally; nothing is written yet.
4. Choose the **Ending reason** (sits between **Mark END** and **Save Rally**; defaults to **`unknown`**), optionally
   type a **Number of shots** (the rally's shot/stroke count ? leave blank to skip), then click
   Save Rally to write one CSV row. The reason **resets to `unknown` after every save** (and the shots field
   clears too) ? so neither silently reuses the previous rally's value, and you can pick the same reason on
   consecutive rallies. Use the **Next rally #** field to resume/insert numbering anywhere.
5. **Recent rallies** list: select a row, then **Edit selected** (loads it back into the fields to fix
   start/end/reason/sport ? **Save changes**) or **Delete selected**. **Undo last** removes the most recent row ?
   the button shows which one, e.g. `Undo last (#7)`, and the status panel shows that row's details ? or clears an
   in-progress mark, or cancels an edit.
6. **Help** button ? usage + an [ending-reason decision guide](docs/ENDING_REASONS.md), rendered inside the dialog.

Output: `<video-stem>.rallies.csv` next to the video (falls back to your home dir if the path can't be
resolved). Re-opening the extension **continues** rally numbering from the existing file ? no duplicate IDs.

Resuming a half-finished video: labels are saved to that CSV as you go. Open the **same video** and enable the
extension and it **reloads your existing rallies** (shown in the Recent list) and continues numbering ? so you can
stop and pick up later. Easiest flow: **open the video first, then enable the extension.** If you enable it *before*
the video is loaded, that's fine too ? the moment you click **Mark START** (the video is playing by then) the tool
switches to that video's own `<video-stem>.rallies.csv` and loads any rallies already saved for it, so nothing is
written to the home-dir fallback by mistake. If you switch videos with the dialog already open, click **Refresh** to
point at the current video and load its rallies. (Playback *position* isn't restored ? scrub to where you stopped.)

> **Where's the help / "About" text?** The empty *"About ?lua"* box under **Tools ?

Plugins and extensions ?
> Add-ons** is VLC's own add-on info dialog; its body ("Lua script") is a constant baked into VLC and **can't be
> set by an extension**. Use the **Help** button inside the Rally Annotator dialog instead. (The descriptor's
> description does show in the *Active Extensions* tab's **"More information"** dialog, and the *Active Extensions*
> list shows the title with its version, e.g. **`Rally Annotator v1.6.3`**, so you can confirm which build is loaded.)

Sports & taxonomy
Net-separated racquet sports share a forced/unforced-error point-stop taxonomy, so one tool covers them all:

Sport	Notes
badminton	the reference sport
tennis	service_fault = fault/double-fault; let supported
table_tennis	let (net serve) supported; fast cadence
pickleball	service_fault for faults; treat "let" per local rules
padel	net-separated; rally boundaries as in tennis

`ending_reason` ? `{unknown, winner, forced_error, unforced_error, service_fault, let, other}` ? a shared
vocabulary across these sports. Keep to these values for clean downstream aggregation.

What the ending reasons mean
`unknown` is the **default** (the field resets to it after every save); every other reason **except `winner`** is
charged to the side that **lost** the rally.

Reason	When the rally ended because?
`unknown`	**default** ? not classified yet. A save with no reason picked records `unknown` (never the previous rally's reason); set a specific reason when you can.
`winner`	the last shot landed **in** and went unreturned (opponent couldn't reach it / only waved). A clean ace counts here.
`forced_error`	the loser **missed** (out or into the net) while **under pressure** ? stretched, rushed, jammed, handling the opponent's pace/spin/depth.
`unforced_error`	the loser **missed a routine shot** they had time **and** position to make, with little or no pressure.
`service_fault`	the point ended **on the serve** ? into the net, out of the service box, illegal action/foot fault, or a double fault.
`let`	the rally is **replayed** under the rules, no point scored (e.g. a tennis/table-tennis serve net-cord that's otherwise good, outside interference).
`other`	none of the above ? occluded footage, injury/retirement, penalty, hindrance. Use sparingly.

The cases people ask about: a ball/shuttle landing **out** is the *hitter's* error ? `forced_error` only if they
were under pressure, otherwise `unforced_error` (it's **never** automatically forced, and never a winner for the
opponent). **Into the net** during a rally is the same; a **serve** into the net is `service_fault`. When unsure
forced-vs-unforced, **default to `unforced_error`**. See the full decision guide with examples and per-sport rules
in **docs/ENDING_REASONS.md** (also available via the in-plugin **Help** button).

Roadmap
- [ ] Live-test the v1.6 dialog in VLC (single Play / Pause toggle, optional Number of shots, two-step Save,
      `unknown`-default reason, editable times, Next rally #, Recent-rallies Edit/Delete) across all five sports.
- [ ] Optional per-sport reason presets / hotkeys.

````
- [ ] Alternative front-ends for power users / remote raters: a `python-vlc` + Tk/Qt app with global keyboard
      shortcuts (S/E/1?6/U), and a zero-install HTML5 `<video>` page that exports the same CSV.
- [x] Optional `shots_count` column (v1.6). Further extra columns (e.g. server/receiver) still TBD behind a toggle.

## Tests
The dialog logic has a headless test suite that stubs VLC's `vlc` API (widgets + playback), loads the real
extension, drives its callbacks, and asserts the results ? run with **Lua 5.1** (the interpreter VLC 3.x embeds):

```bash
lua5.1 test/dialog_test.lua        # exit 0 = pass; covers reason reset, shots column, numbering, undo, help toggle, Play/Pause toggle
```

It covers all the logic but not VLC's GUI rendering or real playback (there's no headless VLC UI automation) ? those
are checked by loading in VLC with `vlc -vv --file-logging` (must scan with no `Error loading` line) plus a manual
click-through. See [test/README.md](test/README.md).

## Contributing
Issues and PRs welcome ? especially live test reports per sport/OS and small UX fixes. The plugin is a single
self-contained Lua file (`vlc/rally_annotator.lua`) using only documented VLC 3.x Lua APIs; please keep
`test/dialog_test.lua` green (`lua5.1 test/dialog_test.lua`) and add a case for new dialog logic.

## License
MIT ? see [LICENSE](LICENSE).

````

13. user (2026-06-20T10:03:25.401Z)

````
# Output CSV format

One row per rally. Header is written once when the file is created.

```
rally_number,start_time,end_time,ending_reason,sport,shots_count
1,8.800,11.500,winner,badminton,9
2,24.389,46.589,unforced_error,badminton,21
3,49.183,54.683,let,badminton,
```

| Column | Type | Notes |
|---|---|---|
| `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
| `start_time` | float | rally start, **decimal seconds** from the video start |
| `end_time` | float | rally end, decimal seconds (always `> start_time`) |
| `ending_reason` | enum | `unknown` (default) / `winner` / `forced_error` / `unforced_error` / `service_fault` / `let` / `other` ? see the decision guide in [ENDING_REASONS.md](ENDING_REASONS.md) |
| `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
| `shots_count` | int | **optional** ? number of shots/strokes in the rally. **Blank** when not entered (as in row 3 above); added in v1.6. Older CSVs without this column still load. |

Notes:
- **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
- Rows are appended live; **Undo last** truncates the last row (one level).
````

````
- Consumers should read by **column name** (the header is stable but column *order* may evolve; extra
  columns may be added later). Most CSV `DictReader`-style loaders already tolerate extra columns.

## Ingesting the labels

This CSV is intentionally compatible with common rally-segmentation tooling that reads
`rally_number,start_time,end_time` by header name and drops rows where `end <= start`:

```bash
# Generic: any tool that takes a labels CSV of start/end seconds.

# Example ? score a detector against these labels, or slice clips by them
# (badminton-highlight-indexer loaders read --labels <this.csv> directly):
python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv --fps 59.94 --frame-width 1920
python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv --rallies all

# Example ? register as curated golden data in sports-data-collector:
python -m src.cli.curate import-local --sport badminton --video-path match.mp4 --annotations-path match.rallies.csv
```

The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.

````

14. user (2026-06-20T10:03:25.553Z)

```
MIT License

Copyright (c) 2026 Avi Dullu

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

```

15. assistant (2026-06-20T10:03:28.831Z)

Now I understand the annotator. Let me check whether the annotator repo has any screenshots/images, and check the train side of things ? what's actually real in the badminton-highlight-indexer for the "train" step (the `worker train` from the recent commits).

16. user (2026-06-20T10:03:31.547Z)

```
---ANY IMAGE DIRS---
docs/CSV_FORMAT.md
docs/ENDING_RALPHS.md
```

17. user (2026-06-20T10:03:31.909Z)

KhelSutra ? Product Overview & Early-User Guide

What it does today, how to get the best out of it, and where it's going.
For field associates and for invited alpha / beta academies, coaches, and players.

In one line

KhelSutra turns ordinary badminton video into useful pieces ? automatically. Put a phone or GoPro
on a tripod behind the court, record a session, and KhelSutra automatically finds the rallies, stitches
a **highlight reel**, and gives you a **simple summary** (how many rallies, how much real play time).
No special hardware, no editing, no manual scrubbing.

The problem we're solving

Academies and serious players already record a lot of video ? and then almost never watch it. Hours of
footage pile up on phones and memory cards because cutting it into useful clips by hand is tedious and
nobody has the time. The good rallies, the moments worth reviewing, stay buried.

Most academy halls also run **several courts at once**, so a single recording is a busy, noisy scene
that ordinary video tools choke on. **Pulling the real rallies out of busy, multi-court footage is the
hard problem KhelSutra is built to crack** ? it already works very well on clean, single-court
recordings, and the multi-court case is exactly what we're sharpening now.

Who it's for

- **Coaching academies & coaches** ? turn raw session footage into review-ready clips and reels for
  teaching, without an editor.
- **Serious players** ? get clips of your own rallies to review, keep, and share.
- Today's focus is **badminton academies in Bengaluru** ? that's who we're building with first.

What it does **today**

This is what's working right now, in the private web app invited users can try:

- **Upload a full session or match video** (from a phone on a tripod, or a GoPro). Long videos are fine.
- **Automatic rally detection** ? it finds the rallies and where they start and end, on its own.
- **Highlight reel** ? the rallies stitched into one MP4 you can watch, download, and share, with a small
  "Generated by Khelsutra.guru" mark.
- **A simple summary** ? how many rallies were found and how much actual play time there was.
- **Re-use & download** ? process once, come back to it later, download the reel.

That's the core loop: **record ? upload ? get back a reel + summary.** No editing skill needed.

> Like any automatic tool, it can occasionally miss a real rally or include one that isn't ? telling us
> when that happens is genuinely useful feedback, not a sign it's broken (more on that below).

How to use it (alpha)

1. **Your KhelSutra associate helps you get access** to the private web app (it's invite-only; today the app is in English).
2. **Upload your session video.** Large files upload in pieces and can take a while on Indian uplinks ? that's normal.
3. **It processes in the background.** You can close the tab and come back later; you don't have to sit and watch it.
4. **When it's done**, you get the **highlight reel + summary** to watch and download.
5. **Re-process anytime.** (Re-running the same video replaces its earlier result rather than keeping both.)

How to get the best out of it (capture matters)

KhelSutra works best on a **steady, full-court recording** ? the same way a coach would set a camera to review play. A few simple habits make a big difference to the results:

- **Put the camera behind the court, slightly elevated**, with the **whole court in frame**.
- **Keep it still** ? a fixed tripod beats a handheld or panning shot every time.
- **Shoot landscape**, with reasonable lighting.
- **Record the whole session** and upload it as-is ? KhelSutra does the cutting.

Footage that fights these habits (shaky handheld, heavy zooming/panning, broadcast-style cuts, very dark halls) is the **hardest case** and where results suffer most. When in doubt: fixed camera, full court, don't touch it. *(Your field contact provides a one-page capture checklist at setup.)*

The part that makes KhelSutra different: **it gets sharper from real academy footage**

This is the heart of the product, and what sets it apart from a generic "AI clipper":

The footage that real academies contribute is what makes KhelSutra better over time ? especially on the busy multi-court halls that are the hard case. When you choose to share your footage, and when coaches confirm or correct what it found, that growing, labelled library is what we use to sharpen the tool. It's designed to improve in new versions as that library grows, rather than stay frozen ? which is why early users aren't just customers, they're co-builders.

Your data stays yours ? and sharing is your choice. We will **never** use your video to improve the tool without your **clear, explicit agreement** ? it is opt-in, never automatic.

Footage involving minors is never used to improve the tool and is never published. We never publish your footage; you decide what, if anything, you share.

How your video is handled (plainly): these are large videos that need real processing, so your video is handled securely on our infrastructure to produce your reel. We don't publish it, and we don't use it to improve the tool without your opt-in.

> **Read this honestly:** today the tool gives you rally detection + reels + summaries. It does **not** retrain itself as you use it, and there is no per-person tracking. The "gets sharper over time" above is the model we're building toward, powered by contributed footage ? not a switch that's already on.

Where it's going ? toward an **AI assistant coach**

Today KhelSutra finds rallies and makes reels. The roadmap ? fed by the growing video library above ? turns it into a genuine training companion:

- **Shot detection** ? recognising smashes, drops, net shots, serves, and clears within each rally.
- **Deeper per-rally and per-match analysis** ? patterns, tendencies, what's working and what isn't.
- **Per-match history** ? track progress over weeks and months, not just one clip.
- **Coaching insights** ? an **AI assistant coach** that supports (never replaces) the real coach, surfacing what to work on next.
- **More sports** ? table tennis next, then other racket/court sports.
- **Your language** ? Kannada and Hindi, alongside English.

> **Read this honestly:** the items in this section are the **direction**, not today's buttons. What you can use *today* is rally detection + highlight reels + the play-time summary. The deeper analysis arrives as the video library grows ? which is exactly why your footage and feedback matter so much.

What feedback helps us most (for alpha users)

You don't need to be technical ? just tell us, in your words:

- **When it missed a real rally**, or **grabbed something that wasn't a rally.**
- Whether the **reel is the right length and order** for how you'd actually use it.
- **What numbers or details you wish the summary showed.**
- Your real **capture conditions** ? how many courts were running, camera/phone, lighting.
- Anything that would make this **genuinely useful for your academy** (or a reason it wouldn't be).

(Send feedback however is easiest ? your KhelSutra associate will help you with that.)

What we're focused on over the next ~4 weeks

1. **Growing the library of real academy footage** ? especially busy **multi-court**

halls (the hard, valuable case). This is the single biggest lever on quality.
2. **Field conversations with academies** ? understanding how coaches and players actually work with video, and what would make this genuinely useful.
3. **Recording sessions and showing academies their own footage** as reels ? *"this is what it looks like for you."*
4. **Sharpening multi-court accuracy** and inviting a few more academies into the private trial.

What KhelSutra is **not** (so expectations stay honest)

- It's an **early, invite-only trial** ? useful, but still improving; you'll hit rough edges.
- It is **not a line-calling, scoring, or umpiring system**, and **not player tracking / identification**.
- **Shot detection and the deeper "AI coach" analytics are on the roadmap, not available today.**
- It is **not a replacement for a coach** ? it's a tool that saves coaches time and adds insight.
- **Pricing isn't set** ? we're still learning what's fair; today's invited use is about getting it right.

Quick answers to common questions

"How accurate is it?"
Very good on a clean, single-court recording. A busy multi-court hall is harder ? and is exactly what we're improving. The best answer is to *show* a sample reel rather than quote a number.

"Do I need a special camera or hardware?"
No. A phone or GoPro on a tripod, behind the court, whole court in frame. We provide a one-page checklist.

"Is my video private? What happens to it?"
We treat your footage as private: we never publish it, and we never use it to improve the tool without your explicit opt-in. These are large videos that need real processing, so your video is handled securely on our infrastructure to make your reel. **Footage involving minors is never used to improve the tool and is never published.**

"Can it tell me the score / who won / call lines?"
Not today. Today it finds rallies and builds a highlight reel + a play-time summary.

"Does it work for doubles and tournaments?"
It finds rallies in doubles too. Tournament footage varies: a clean, fixed, full-court recording works well; crowded multi-court tournament halls are the hard case we're specifically working on.

"Will it get better?"
Yes ? that's the whole idea. The more real footage runs through it (with permission) and the more coaches correct it, the sharper future versions get, and the sooner shot detection and deeper analysis arrive.

"Which sports?"
Badminton today; table tennis and other racket sports are on the roadmap.

```
**"When can I use it / how do I join?"**
It's an invite-only private trial. If you're interested, the best next step is to let us record a session
and show you your own court and players as a reel.

---

*KhelSutra · khelsutra.guru · Internal & invited-trial overview, June 2026.*
*Field associates: the role description is in the Field Associate flyer, and the conversation script and
visit rules are in the separate Field Visit Playbook.*
```

18. assistant (2026-06-20T10:03:35.633Z)

Important finding: the annotator repo has NO screenshots/images committed. So any screenshot plan must say "to-capture" ? I cannot point at images that exist in the repo. Let me verify the train side reality (the `worker train` Gen-0 mock) and confirm whether there's any existing Studio-UI in the site. Let me check the khelsutra site structure and the worker train command.

19. user (2026-06-20T10:03:39.507Z)

```
Already up to date.
===SITE PUBLIC===
site/public/capture.html
site/public/index.html
site/public/og.png
===STUDIO/UI refs===
```

20. user (2026-06-20T10:03:39.634Z)

```
Found 1 file
backend\worker\__main__.py
```

21. assistant (2026-06-20T10:03:43.067Z)

So the khelsutra site has only `index.html` + `capture.html` + `og.png` ? no Studio UI yet, and there is no annotator/train page. This confirms the Studio-UI pages are net-new [design]. Let me verify the actual state of the train side (the Gen-0 worker train) to characterize the "train" step honestly.

22. user (2026-06-20T10:03:43.290Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""
from __future__ import annotations
```

```python

import argparse
import csv
import json
import os
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
        try:
            return cast(v)
        except ValueError:
            pass
    return v


def _apply_overrides(config: Any, sets: List[str]) -> None:
    """Apply ``a.b.c=value`` overrides onto the typed config (e.g. indexing.skip_ai_handoff=true)."""
    for kv in sets or []:
        key, sep, val = kv.partition("=")
        if not sep:
            raise ValueError(f"--set expects k=v, got {kv!r}")
        parts = key.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p)
        setattr(obj, parts[-1], _coerce(val))


def _write_intervals_csv(segments: List[dict], path: str) -> None:
    """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-friendly
    artifact (same schema as the rally-annotator labels)."""
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["start", "end", "shot_count", "ending_reason"])
        for s in segments:
            w.writerow([
                f'{float(s.get("start_time", 0.0)):.3f}',
                f'{float(s.get("end_time", 0.0)):.3f}',
                s.get("shot_count", ""),
                s.get("ending_reason", s.get("shot_count_confidence", "")),
            ])


def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
```

```
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg)
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI).
        val_results, val_context = validator_registry.run_all_with_context(local_video, config)
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(video_id, local_video, "failed" if failures else "processed",
                     [r.model_dump() for r in val_results])

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print("[worker] validation FAILED: "
                      + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
                return 2
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[worker] unknown segmenter: {seg_name!r} "
                      f"(have {list(segmenter_registry.list_available())})")
                return 2
            segmenter_actual = seg_name
            result = segmenter_cls(db, config).process_video(video_id, local_video, max_frames=args.max_frames)
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return 3
            segments = result.value if hasattr(result, "value") else result
            status = "success"
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the main CLI.
            emit_indexer_report(
```

```
            db=db, video_id=video_id, video_path=local_video, config=config,
            val_results=val_results, val_context=val_context,
            segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
            was_reingest=False, segmentation_ran=segmentation_ran,
        )

    # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
    out_local = os.path.join(scratch, "outputs", video_id)
    os.makedirs(out_local, exist_ok=True)
    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    _write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))

    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(out_local)):
        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
        pushed.append(uri)

    print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0


#: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
#: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
_VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")


def _probe_video_fps_width(path: str):
    """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

    The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a WRONG
    fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles every time).
    cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for the
    inference report (it flags the fallback) but corrupting for REAL training data. So here we
    probe with ffprobe (same tool as ``get_video_duration``) and return None so the caller SKIPS
    rather than guesses.
    """
    import json
    import subprocess
    try:
        out = subprocess.check_output(
            ["ffprobe", "-v", "error", "-select_streams", "v:0",
             "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
            stderr=subprocess.DEVNULL).decode()
        st = (json.loads(out).get("streams") or [{}])[0]
        num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
        fps = float(num) / float(den or "1")
        width = float(st.get("width") or 0)
        if fps > 0 and width > 0:
            return fps, width
    except (subprocess.SubprocessError, ValueError, KeyError, OSError, json.JSONDecodeError):
        pass
```

```
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
            return None
        try:
            runner, _ = seg._resolve_runner(config.get("indexing", {}))
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
            shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
          * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
            DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
            is the M3.5 "first real training" path; only the trajectory source flips.
        """
        import subprocess

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()
```

```

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
    labels_dir = os.path.join(scratch, "labels")
    traj_dir = os.path.join(scratch, "trajectories")
    videos_dir = os.path.join(scratch, "videos")
    os.makedirs(labels_dir, exist_ok=True)
    os.makedirs(traj_dir, exist_ok=True)

    corpus = args.corpus_uri.rstrip("/")
    label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
                        if u.lower().endswith(".csv"))
    if len(label_uris) < 2:
        print(f"[worker] need >=2 labeled videos for leave-one-video-out; found {len(label_uris)} "
              f"under {corpus}/labels/")
        return 2

    # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
    runner = None
    video_by_stem: dict = {}
    if args.trajectory_source == "detector":
        os.makedirs(videos_dir, exist_ok=True)
        runner = _resolve_train_detector(args, config)
        if runner is None:
            return 2
        for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
            vname = storage.parse_uri(vuri).name
            if not vname.lower().endswith(_VIDEO_EXTS):
                continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow the video
            video_by_stem[os.path.splitext(vname)[0]] = vuri

    print(f"[worker] trajectory source: {args.trajectory_source}")
    specs: List[dict] = []
    for uri in label_uris:
        fname = storage.parse_uri(uri).name                    # e.g. GX020094_proxy.rallies.csv
        name = fname.replace(".rallies.csv", "").replace(".csv", "")
        local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
        if args.trajectory_source == "detector":
            vuri = video_by_stem.get(name)
            if not vuri:
                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
                continue
            local_video = storage.fetch(vuri, os.path.join(videos_dir, storage.parse_uri(vuri).name),
                                        config=storage_cfg)
            video_id = compute_video_id(local_video)
            # Real fps/width drive the trajectory frame->time mapping; PROBE (don't guess) ? a
            # wrong fps silently mis-aligns every label, so skip the video if ffprobe can't read it.
            meta = _probe_video_fps_width(local_video)
            if meta is None:
                print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ? skipping (won't guess).")
                continue
            fps, frame_width = meta
            traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
            if not traj:
                print(f"[worker] detector produced no trajectory for {name!r} ?
```

```
skipping.")
                continue
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": fps, "frame_width": frame_width})
        else:
            from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
            traj = os.path.join(traj_dir, f"{name}_traj.csv")
            synth_trajectory_from_labels(local_label, traj, fps=args.fps, frame_width=args.frame_width)
            specs.append({"name": name, "trajectory": traj, "labels": local_label,
                          "fps": args.fps, "frame_width": args.frame_width})

    if len(specs) < 2:
        print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got {len(specs)}.")
        return 2
    manifest_path = os.path.join(scratch, "manifest.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(specs, f, indent=2)
    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg)
                       if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="python -m backend.worker",
                                description="Storage-aware worker: pull ? run pipeline ? push.")
    sub = p.add_subparsers(dest="cmd", required=True)

    pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
    pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// / gcs://")
    pi.add_argument("--out-uri", required=True, help="output prefix (outputs/<video_id>/? is appended)")
    pi.add_argument("--segmenter", default=None, help="default: indexing.default_segmenter")
```

```
    pi.add_argument("--config", default=None, help="config.json path (default: ./config.json)")
    pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp dir)")
    pi.add_argument("--max-frames", type=int, default=None)
    pi.add_argument("--set", action="append", default=[], metavar="k=v",
                    help="config override, e.g. --set indexing.skip_ai_handoff=true")
    pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)")
    pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+ videos/ for "
                    "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
    pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/ is appended)")
    pt.add_argument("--version", required=True, help="semver for the artifact bundle")
    pt.add_argument("--trajectory-source", choices=["synth", "detector"], default="synth",
                    help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
                         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
                         "(needs videos/ in the corpus; honours detector_impl).")
    pt.add_argument("--segmenter", default="wasb_hybrid",
                    help="trajectory hybrid whose detector is used for --trajectory-source detector")
    pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or storage URI)")
    pt.add_argument("--config", default=None)
    pt.add_argument("--scratch", default=None)
    pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps (detector probes the video)")
    pt.add_argument("--frame-width", type=float, default=1280.0,
                    help="synth trajectory frame width (detector probes the video)")
    pt.add_argument("--set", action="append", default=[], metavar="k=v")
    pt.set_defaults(func=cmd_train)
    return p


def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    return args.func(args)


if __name__ == "__main__":
    sys.exit(main())
```

23. assistant (2026-06-20T10:03:52.690Z)

Now I have a precise, honest picture of the train side. Key honesty anchors:
- `train --trajectory-source synth` is the default and is a CPU-mock (no GPU). `detector` is the real WASB path, which is GPU/native-WASB-gated.
- The whole thing is a CLI (`python -m backend.worker train`), not a Studio button. The Gen-0 ship decision is still owner-gated per the plan (Phase 1).
- The annotator is a separate VLC Lua plugin, CSV-out, MIT-licensed, no in-browser annotator.

Let me confirm the CSV column join: annotator emits `rally_number,start_time,end_time,ending_reason,sport,shots_count`; the collector ingests via `python -m src.cli.curate import-local`; and the Roora guideline CSV is a richer superset. Let me also quickly check the ENDING_REASONS doc exists and whether the site has any "GitHub" links already, then I have enough.

24. user (2026-06-20T10:03:54.282Z)

No matches found

25. assistant (2026-06-20T10:04:06.288Z)

Confirmed: the current site has zero mention of GitHub, the annotator, or "train your own model." So the Studio-UI pages are entirely net-new design. I now have everything I need to produce an honest, fully-grounded answer.

Let me capture the verified reality before composing the schema:

Verified [verified] facts:
- `rally-annotator` is a public GitHub repo at https://github.com/avidullu/rally-annotator, MIT-licensed, a single VLC 3.0.x Lua extension (`vlc/rally_annotator.lua`), with README + `docs/CSV_FORMAT.md` + `docs/ENDING_REASONS.md`.
- It outputs `<video-stem>.rallies.csv` with columns `rally_number,start_time,end_time,ending_reason,sport,shots_count`.
- It supports 5 net-separated sports; `ending_reason` taxonomy is the shared vocabulary.
- The repo has **no committed screenshots/images** (only docs markdown) ? so screenshots must be captured fresh, never fabricated.
- The CSV ingests into `sports-data-collector` via `python -m src.cli.curate import-local` and into the indexer's eval loaders.
- The indexer has `python -m backend.worker train` (Gen-0 train-from-bucket); default `--trajectory-source synth` is a CPU mock; `detector` is the real GPU/native-WASB path.
- KhelSutra guardrails: paid LLMs never a training source; MIT/Apache-2.0/BSD only; minors excluded; per-user training data needs explicit opt-in; gait/biometrics deferred.
- The live site (index.html) already has honesty sections ("Read this honestly") and a "Your data stays yours / Minors are protected" block.

[design] (not built): any Studio UI page, in-app annotator, a one-click "train" button, a model registry UI.

[researched]: the owned-model roadmap (TAL head now ? VLM later, Qwen3-VL) is plan/roadmap, owner-gated.

Now I'll return the structured output.

26. user (2026-06-20T10:05:12.963Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 12256 bytes): {"usp_framing": "POSITIONING (one line): \"Annotate your own footage and train a model that's YOURS ? on a stack you can read, on hardware you control.\" This is the bring-your-own-model moat, and it
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

27. assistant (2026-06-20T10:05:18.442Z)

The tool expects structured fields, not a raw wrapper. Let me call it with the proper field structure.

28. user (2026-06-20T10:06:32.671Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 13666 bytes): {"usp_framing": "POSITIONING (one line): \"Annotate your own footage and train a model that's YOURS ? on an open, readable stack you control.\" This is the bring-your-own-model moat, and it is honestl
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

29. assistant (2026-06-20T10:06:37.152Z)

The harness is wrapping my input oddly. Let me pass the fields directly as native tool parameters (not as a JSON string), keeping the prose plain ASCII to avoid escaping issues.

30. user (2026-06-20T10:07:42.528Z)

Output does not match required schema: root: must have required property 'setup_help', root: must have required property 'screenshot_plan', root: must have required property 'honest_caveats', root: must have required property 'file_refs'

31. assistant (2026-06-20T10:07:46.752Z)

The named parameters after the first one were treated as a single block. I need to send all fields as proper named tool parameters. Let me re-issue with each parameter explicitly named (the issue was the `<parameter name=` tags after usp_framing got merged). I'll send them as a clean set.

32. user (2026-06-20T10:08:35.168Z)

Output does not match required schema: root: must have required property 'setup_help'

33. assistant (2026-06-20T10:08:41.225Z)

The first parameter block (usp_framing) seems to have absorbed the rest because of an unescaped issue. The error now only complains about `setup_help` missing ? which means screenshot_plan, honest_caveats, and file_refs were accepted but setup_help was not parsed. The likely culprit is the `\\\\` in the setup_help array (over-escaped backslashes inside an array string). Let me resend with single backslash usage simplified ? avoid backslashes entirely in the path examples to sidestep escaping.

34. user (2026-06-20T10:09:27.285Z)

Output does not match required schema: root: must have required property 'setup_help'

35. assistant (2026-06-20T10:09:31.222Z)

Only `setup_help` is failing now while the other three array fields pass. The difference: setup_help strings contain `<` and `>` characters (e.g. `<video-stem>`, `<sport>`, `<bucket>`). Those angle brackets may be tripping the parser as it tries to interpret tool-call markup. Let me replace all `<...>` placeholders with brace or word forms in setup_help.

36. user (2026-06-20T10:10:18.783Z)

Output does not match required schema: root: must have required property 'setup_help'

37. assistant (2026-06-20T10:10:22.832Z)

Still failing on `setup_help` only. The other three arrays parse fine. The remaining difference: setup_help strings contain `>` characters (e.g. `Tools > Plugins`, `View menu > Rally Annotator`, `record-upload-reel`). Wait ? screenshot_plan also has `>`... Actually it doesn't. Let me remove every `>` from setup_help and replace with words.

38. user (2026-06-20T10:11:10.537Z)

Output does not match required schema: root: must have required property 'setup_help'

39. assistant (2026-06-20T10:11:15.217Z)

Still only `setup_help`. The angle brackets are gone, the `>` are gone. The other arrays parse. The remaining structural difference is that setup_help is the longest array with the most special chars. Let me try a minimal `setup_help` (short strings, plain ASCII, no slashes/dots-heavy tokens) to isolate whether it's a length/content issue versus something positional, while keeping the others. If a minimal version passes, I'll know it's content and can reintroduce detail carefully.

40. user (2026-06-20T10:11:50.867Z)

Output does not match required schema: root: must have required property 'setup_help'

41. assistant (2026-06-20T10:11:55.788Z)

Persistently only `setup_help` fails even when simplified. The other three arrays (screenshot_plan, honest_caveats, file_refs) consistently pass. This strongly suggests the harness is dropping the `setup_help` parameter entirely ? likely because it is the parameter immediately following the very large `usp_framing` text block, and the boundary between them isn't being parsed. Let me reorder: put `setup_help` BEFORE `usp_framing` so the large block isn't adjacent to it, and shrink `usp_framing` slightly.

42. user (2026-06-20T10:12:30.019Z)

Structured output provided successfully

43. assistant (2026-06-20T10:12:41.252Z)

The structured output was accepted. (The repeated failures were a parser issue with the `setup_help` field when it followed the large `usp_framing` text block; reordering fields so `setup_help` came first resolved it.)

Summary of what I delivered, grounded in the actual repos:

- **USP framing** ? "annotate your footage, train a model that's yours" as a bring-your-own-model moat, tied to four guardrails that already exist (commercial-clean MIT/Apache licensing, paid-LLM-never-a-training-source, privacy/consent, minors-excluded), and explicitly NOT claiming accuracy superiority over paid LLMs (no such benchmark exists).
- **Setup help** ? the end-to-end flow: install the VLC plugin ? record ? annotate in VLC ? the plugin auto-writes the `.rallies.csv` ? register with `curate import-local` ? `worker train` (synth CPU-mock default vs GPU/native-WASB `detector`) ? use the model, with reality tags per step.
- **Screenshot plan** ? flagged the cardinal finding that the `rally-annotator` repo has **zero committed images**, so every screenshot is to-capture (never fabricated), plus which repo text can be embedded verbatim.
- **Honest caveats** ? annotator is a separate VLC tool (not in-browser), Studio pages are net-new design, train is GPU-gated and owner-gated, CLI/manual steps throughout, two distinct CSV schemas, VLC 3.0.x-only.
- **File refs** ? the verified source files in `rally-annotator`, the khelsutra site, and the indexer docs/worker.

One note worth flagging for you directly: the live `index.html` already carries the privacy/consent/minors honesty language, but has **no** GitHub link, no annotator mention, and no "train your own model" content ? so the Studio pages are genuinely net-new and the USP currently isn't represented anywhere on the public site.